



UCBM
UNIVERSITA' CAMPUS BIO-MEDICO DI ROMA



Innovazione e trasformazione digitale

A.A. 2025-26

Modulo 2: Digitalizzazione e ottimizzazione nel settore dell'energia elettrica

Prof. Francesco Conte

Smart management examples



UCBM
UNIVERSITA' CAMPUS BIO-MEDICO DI ROMA



Model Predictive Control

Model Predictive Control

Basic Concepts



Let us consider the optimal control problem for a linear discrete-time system:

$$\{u^*(j)\}_{j=0}^{T-1} = \arg \min_{\{u(j)\}_{j=0}^{T-1}} J(x(0), \{x(j)\}_{j=1}^T, \{u(j)\}_{j=0}^{T-1})$$

$$\text{such that } x(j+1) = Ax(j) + Bu(j) + Fd(j) \quad \forall j \in [0, T-1]$$

where:

- $x(j) \in \mathbb{R}^n$ is the systems state vector;
- $u(j) \in \mathbb{R}^p$ is the control input vector;
- $d(j) \in \mathbb{R}^d$ is the disturbance vector: $d(j) = \hat{d}(j) + \varepsilon(j)$;
- A, B, F are modeling matrices with suitable dimensions;
- J is the control cost function.

The state $x(0)$ is supposed to be known (state is accessible).

Typical cost functions:

- linear $J = \sum_{j=0}^T c^T x(j) + \sum_{j=0}^{T-1} b^T u(j)$
- quadratic:

$$J = \sum_{j=0}^{T-1} [(x(j) - x^*)^T P (x(j) - x^*) + u(j)^T Q u(j)] + (x(T) - x^*)^T S (x(T) - x^*)$$

Model Predictive Control

Basic Concepts



In the particular case with quadratic cost function and infinite time horizon:

$$\{u^*(j)\}_{j=0}^{T-1} = \arg \min_{\{u(j)\}_{j=0}^{T-1}} \sum_{j=0}^{\infty} [(x(j) - x^*)^T P (x(j) - x^*) + u(j)^T Q u(j)] + (x(T) - x^*)^T S (x(T) - x^*)$$

$$\text{such that } x(j+1) = Ax(j) + Bu(j) + Fd(j) \quad \forall j \in [0, \infty)$$

if (A, B) is a controllable couple, and without considering the disturbance ($F = 0$), there exists a gain K (function of P, Q, S, A and B) such that

$$u^*(j) = -Kx(j)$$

is the optimal *closed-loop* optimal solution.

Model Predictive Control

Basic Concepts



In the general *finite-time* and *constrained* case:

$$\{u^*(j)\}_{j=0}^{T-1} = \arg \min_{\{u(j)\}_{j=0}^{T-1}} J(x(0), \{x(j)\}_{j=1}^T, \{u(j)\}_{j=0}^{T-1})$$

$$\begin{aligned} \text{such that } x(j+1) &= Ax(j) + Bu(j) + Fd(j) & \forall j \in [0, T-1] \\ x_{\min} &\leq x(j) \leq x_{\max} & \forall j \in [1, T] \\ u_{\min} &\leq u(j) \leq u_{\max} & \forall j \in [0, T-1] \\ Mx(j) + Lu(i) &\leq b & \forall j \in [1, T], i \in [0, T-1] \\ \bar{M}x(j) + \bar{L}u(i) &= \bar{b} & \forall j \in [1, T], i \in [0, T-1] \end{aligned}$$

there not exists a closed loop closed form solution.

We can solve the optimization problem using any solver, but the solution will be *open-loop*.

Model Predictive Control

Receding Horizon Principle



Model Predictive Control (MPC) adopts the *receding horizon principle*, by which the solution of the optimal control problem becomes closed loop.

Let us indicate with $k = 0, 1, \dots$ the discrete time step. Given a *time horizon* T , we suppose to know at time k the current state $x(k)$ and a forecast for the disturbance $\{\hat{d}(j)\}_{j=k}^{T-1}$.

The receding horizon principle consists of **(1) solving at time k the optimization problem**

$$\{u^*(j)\}_{j=k}^{k+T-1} = \arg \min_{\{u(j)\}_{j=k}^{T-1}} J(x(k), \{x(j)\}_{j=k+1}^{k+T}, \{u(j)\}_{j=k}^{k+T-1})$$

$$\begin{aligned} \text{such that} \quad & x(j+1) = Ax(j) + Bu(j) + Fd(j) & \forall j \in [k, k+T-1] \\ & x_{\min} \leq x(j) \leq x_{\max} & \forall j \in [k+1, k+T] \\ & u_{\min} \leq u(j) \leq u_{\max} & \forall j \in [k, k+T-1] \\ & Mx(j) + Lu(i) \leq b & \forall j \in [k+1, T], i \in [k, k+T-1] \\ & \bar{M}x(j) + \bar{L}u(i) \leq \bar{b} & \forall j \in [k+1, T], i \in [k, k+T-1] \end{aligned}$$

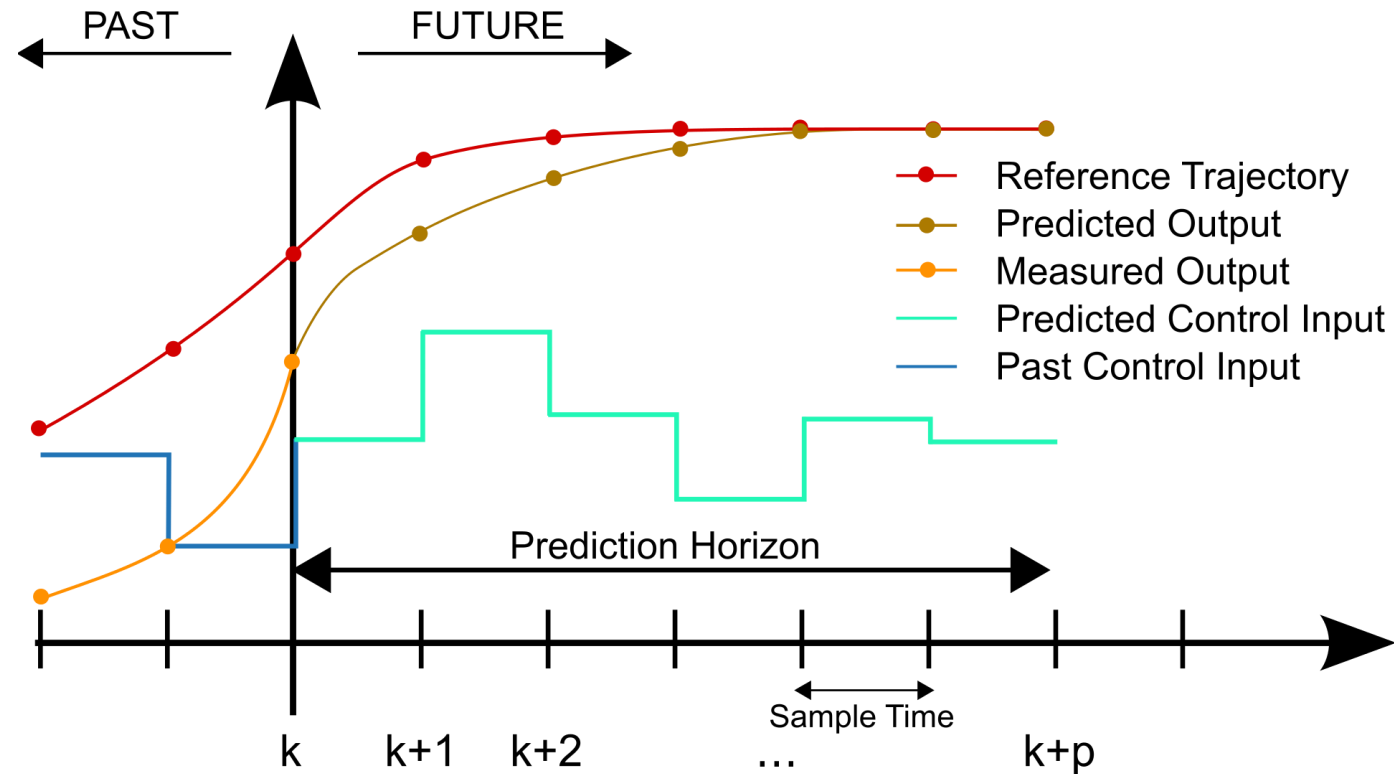
(2) apply the control $u^*(k)$ (first element of the optimal control trajectory), and **(3) repeat the same procedure at time $k+1$.**

Model Predictive Control

Receding Horizon Principle



Thanks to the receding horizon principle, MPC is a closed-loop control technique, meaning that it is robust with respect to measurements and modeling errors.



Model Predictive Control

Possible Forms



- With different state and control time horizons:

$$\{u^*(j)\}_{j=k}^{k+T_c-1} = \arg \min_{\{u(j)\}_{j=k}^{T_c-1}} J \left(x(k), \{x(j)\}_{j=k+1}^{k+T_s}, \{u(j)\}_{j=k}^{k+T_c-1} \right)$$

$$\begin{aligned} \text{such that } x(j+1) &= Ax(j) + Bu(j) + Fd(j) & \forall j \in [k, k+T-1] \\ x_{min} &\leq x(j) \leq x_{max} & \forall j \in [k+1, k+T] \\ u_{min} &\leq u(j) \leq u_{max} & \forall j \in [k, k+T-1] \\ Mx(j) + Lu(i) &\leq b & \forall j \in [k+1, T], i \in [k, k+T-1] \\ \bar{M}x(j) + \bar{L}u(i) &\leq \bar{b} & \forall j \in [k+1, T], i \in [k, k+T-1] \\ u(j) &= u(k+T_c-1) & \forall j \in [k+T_c, k+T_s-1] \end{aligned}$$

where T_c is the control time horizon and T_s is the state time horizon, such that $T_s > T_c$.

Model Predictive Control

Possible Forms



- Optimal tracking of a reference output signal:

$$\{u^*(j)\}_{j=k}^{k+T-1} = \arg \min_{\{u(j)\}_{j=k}^{k+T-1}} \sum_{j=k}^{k+T-1} \left[(y(j) - y^*(j))^T P (y(j) - y^*(j)) + u(j)^T Q u(j) \right] + (y(T) - y^*(T))^T S (y(T) - y^*(T))$$

$$\begin{aligned} \text{such that } x(j+1) &= Ax(j) + Bu(j) + Fd(j) & \forall j \in [k, k+T-1] \\ y(j) &= Cx(j) & \forall j \in [k+1, k+T] \\ x_{\min} &\leq x(j) \leq x_{\max} & \forall j \in [k+1, k+T] \\ u_{\min} &\leq u(j) \leq u_{\max} & \forall j \in [k, k+T-1] \\ Mx(j) + Lu(i) &\leq b & \forall j \in [k+1, T], i \in [k, k+T-1] \\ \bar{M}x(j) + \bar{L}u(i) &\leq \bar{b} & \forall j \in [k+1, T], i \in [k, k+T-1] \end{aligned}$$

Model Predictive Control

Possible Forms



- If the cost function and/or the constraints are nonlinear, we have **nonlinear MPC**.
- If some state and/or control variables are integer, we have **mixed-integer MPC**.
- If we take into account the uncertainties in the state variable dynamics, we have **stochastic MPC**.
- If the state is not accessible, some state observer or filter should be used.

Model Predictive Control

Example 1



Simplified room model for air temperature regulation (cooling modality):

$$T(k + 1) = \alpha T(k) - \beta R \eta P_l(k) + \beta T_{ex}(k)$$

where:

- $\alpha = \exp\left(-\frac{\Delta t_s}{CR}\right)$ and $\beta = 1 - \alpha$;
- R is the thermal resistance of the walls [$^{\circ}\text{C}/\text{kW}$];
- C is the thermal capacitance of the air [$\text{kWh}/^{\circ}\text{C}$] (energy required to vary the temperature of 1°C);
- $T_{ex}(k)$ external temperature [$^{\circ}\text{C}$] (measured or forecasted);
- η cooling efficiency (>1) [pu].

Building data: $R = 1.9^{\circ}\text{C}/\text{kW}$, $\eta = 1.90$ (COP >1), $C = 3.925$ [$\text{kWh}/^{\circ}\text{C}$].

Control specifics:

Objective: keep the temperature as close as possible to the set-point $T^{sp} = 22^{\circ}\text{C}$ with a tolerance of $\Delta = 2^{\circ}\text{C}$.

Assumptions: P_l can be set with continuity within the interval $[0 \quad P_l^{nom}]$, with $P^{nom} = 4$ kW, $\Delta t_s = 1$ h,
 $T(k)$ and $T_{ex}(k)$ are measured.

Model Predictive Control

Example 1



We solve the problem adopting MPC with the following formulation:

$$\{P_l^*(j)\}_{j=k}^{k+T-1} = \arg \min_{\{P_l(j)\}_{j=k}^{k+T-1}} \sum_{j=k+1}^{k+T-1} w_T (T(j) - T^{sp})^2 + w_T^f (T(k+T) - T^{sp})^2 + \sum_{j=k}^{k+T-1} w_P P_l(j)^2$$

$$\text{such that } T(j+1) = \alpha T(j) - \beta R \eta P_l(j) + \beta T_{ex}(j) \quad \forall j \in [k, k+T-1]$$

$$0 \leq P_l(j) \leq P_l^{nom} \quad \forall j \in [k, k+T-1]$$

$$T(j) \leq T^{sp} + \Delta \quad \forall j \in [k+1, k+T]$$

$T(k)$ is available since it is measured.

Since we have not forecast for T_{ex} , we adopt the simplest assumption that $T_{ex}(j) = T_{ex}(k) \quad \forall j \in [k+1: k+T-1]$

Model Predictive Control

Example 1



The solution is implemented in the MATLAB script “example2.m” and the Python file “example2.py”

FILE example2.py

```
def mpc_step(Tk, Tex_forecast, parms):
    parms = SimpleNamespace(**parms)
    model = ConcreteModel() #define the pyomo model

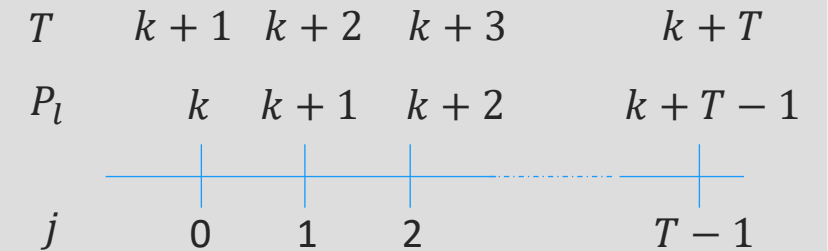
    # Sets
    model.J = RangeSet(0, parms.T-1)    # time index

    # Variables
    model.Pl = Var(model.J, domain=NonNegativeReals)
    model.T = Var(model.J, domain=Reals)

    # Define objective function
    def obj_rule(model):
        return sum(parms.wT*(model.T[j]-parms.Tsp)**2 for j in model.J if j < parms.T-1)
            + parms.wTf*(model.T[parms.T-1]-parms.Tsp)**2 + sum(parms.wP*(model.Pl[j])**2 for j in model.J)

    model.obj = Objective(rule=obj_rule, sense=minimize)

    ...
```



Model Predictive Control

Example 1



The solution is implemented in the MATLAB script “example2.m” and the Python file “example2.py”

FILE example2.py

...

```
# Constraints
def con1(model, j):
    if j==0:
        return model.T[j] == parms.alpha*Tk-parms.beta*parms.R*parms.eta*model.Pl[j]+parms.beta*Tex_forecast[j];
    else:
        return model.T[j] == parms.alpha*model.T[j-1]-parms.beta*parms.R*parms.eta*model.Pl[j]+parms.beta*Tex_forecast[j];

def con2(model, j):
    return model.Pl[j] <= parms.Pnom

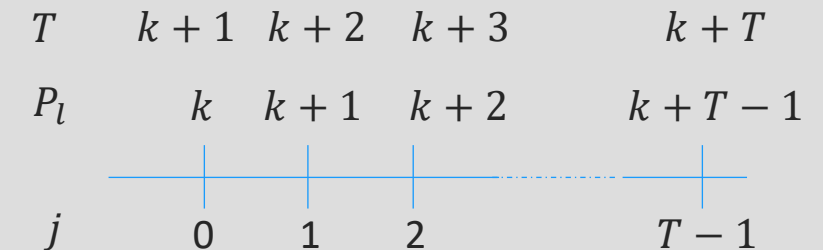
def con3(model, j):
    return model.T[j] <= parms.Tsp+parms.Delta

# Add constraints to model
model.con_1 = Constraint(model.J, rule=con1)
model.con_2 = Constraint(model.J, rule=con2)
model.con_3 = Constraint(model.J, rule=con3)

# solve
results = SolverFactory('gurobi').solve(model)

out = {"PL":value(model.Pl[0]), #applying receding horizon
       "solver_status": results.solver.status == SolverStatus.ok,
       "opt_condition": results.solver.termination_condition == 'optimal'
      }

return out
```

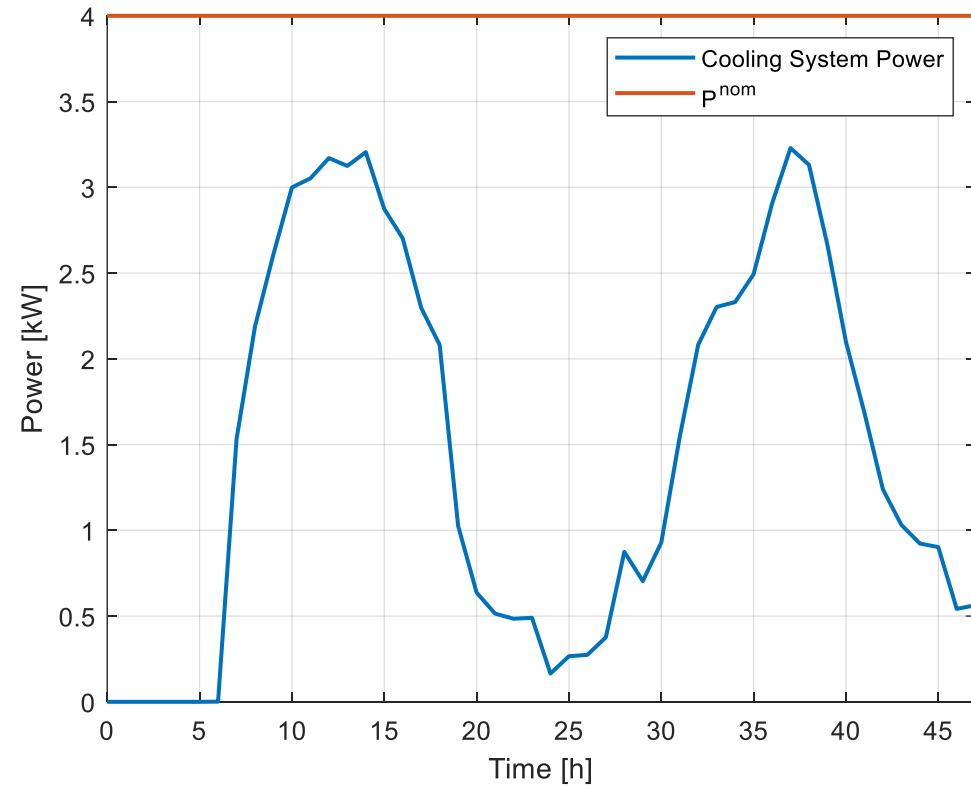
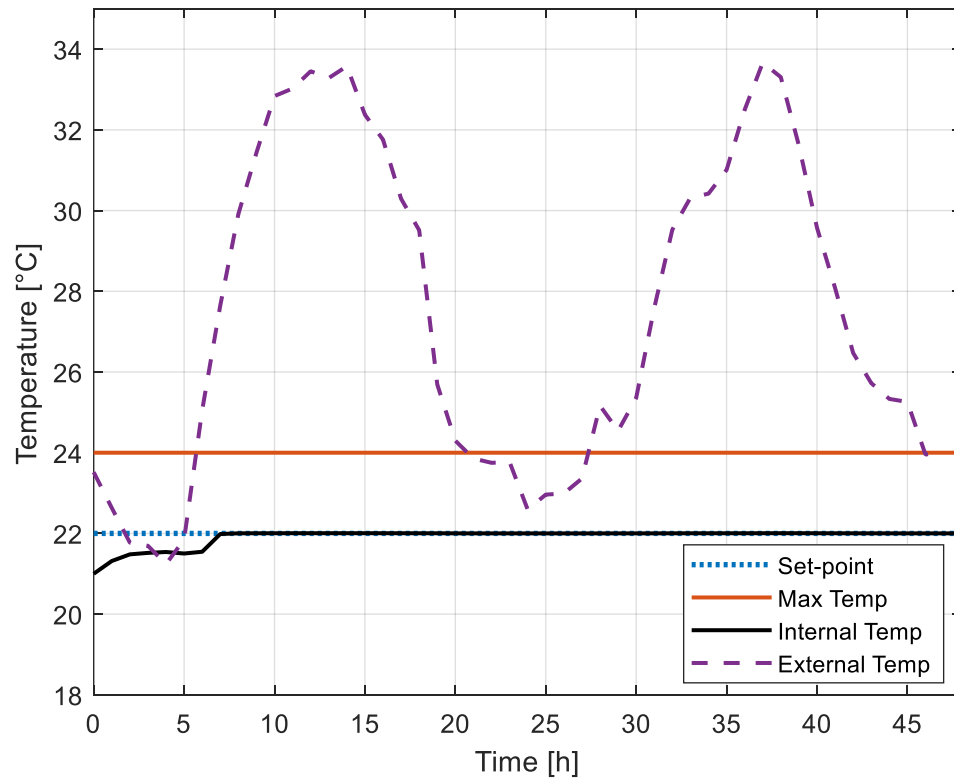


Model Predictive Control

Example 1



Optimization weights: $w_T = w_T^f = 1, w_P = 0.1/(P^{nom})^2$ **PRIORITY TO MINIMIZE ERROR FROM THE TEMPERATURE SET-POINT**
Control horizon: $T = 5$

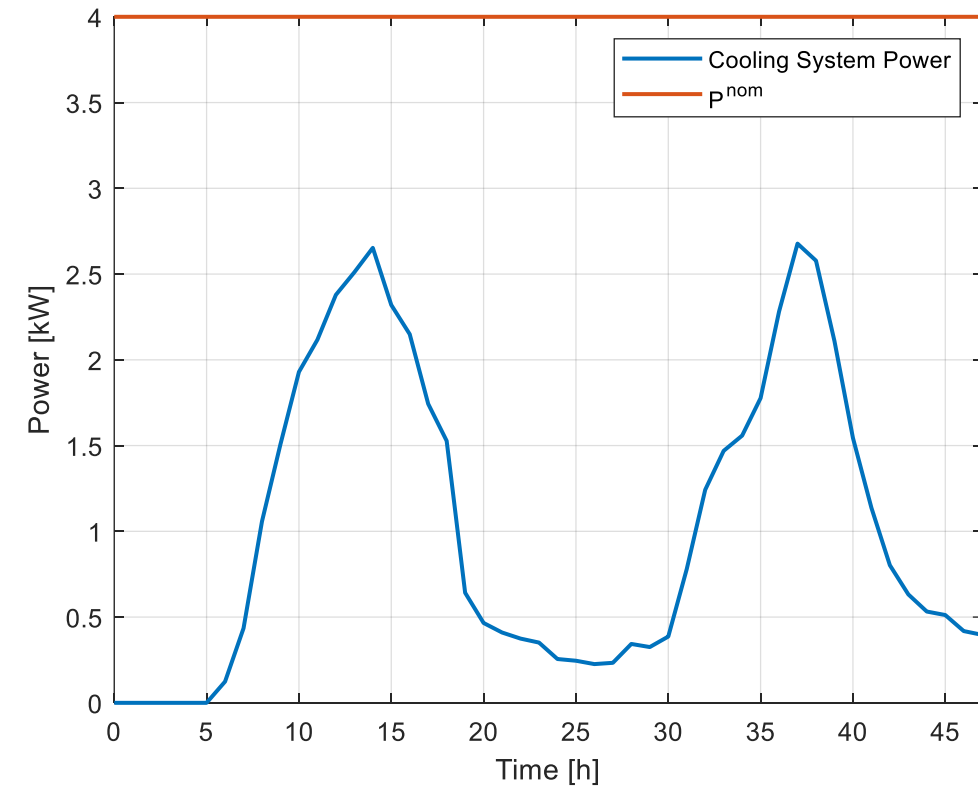
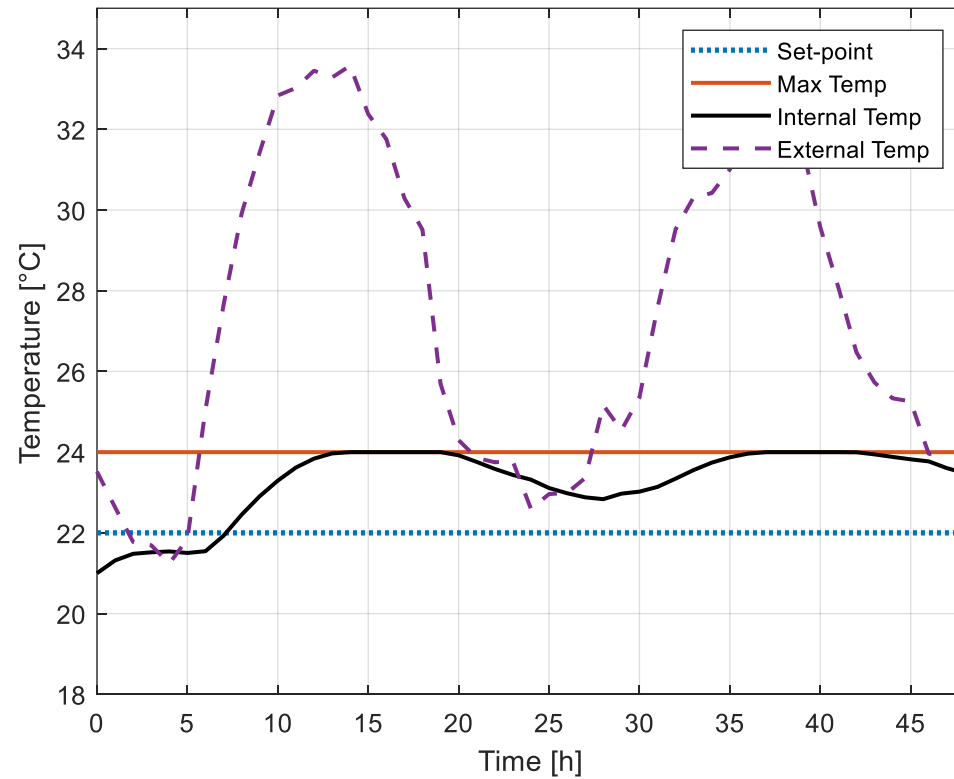


Model Predictive Control

Example 1



Optimization weights: $w_T = w_T^f = 0.01, w_P = 1/(P^{nom})^2$ **PRIORITY TO MINIMIZE POWER CONSUMPTION**
Control horizon: $T = 5$



Model Predictive Control

Example 2



Simplified room model for air temperature regulation (cooling modality):

$$T(k+1) = \alpha T(k) - \beta R \eta P_l(k) + \beta T_{ex}(k)$$

where:

- $\alpha = \exp\left(-\frac{\Delta t_s}{CR}\right)$ and $\beta = 1 - \alpha$;
- R is the thermal resistance of the walls [$^{\circ}\text{C}/\text{kW}$];
- C is the thermal capacitance of the air [$\text{kWh}/^{\circ}\text{C}$] (energy required to vary the temperature of 1°C);
- $T_{ex}(k)$ external temperature [$^{\circ}\text{C}$] (measured or forecasted);
- η cooling efficiency (>1) [pu].

Building data: $R = 1.9^{\circ}\text{C}/\text{kW}$, $\eta = 1.90$ (COP >1), $C = 3.925$ [$\text{kWh}/^{\circ}\text{C}$]

Control specifications:

Objective: keep the temperature as close as possible to the set-point $T^{sp} = 22^{\circ}\text{C}$ with a tolerance of $\Delta = 2^{\circ}\text{C}$ and minimize the cost of energy consumption.

Assumptions: P_l can be set with continuity within the interval $[0 \quad P_l^{nom}]$, with $P^{nom} = 4$ kW, $\Delta t_s = 1$ h.

$T(k)$ and $T_{ex}(k)$ are measured;

at time k we are provided with a forecast of T_{ex} for the next 24 h;

energy costs at any time $c(k)$ are known.

Model Predictive Control

Example 2



We solve the problem adopting MPC with the following formulation

$$\{P_l^*(j)\}_{j=k}^{k+T-1} = \arg \min_{\{P_l(j)\}_{j=k}^{k+T-1}} \sum_{j=k}^{k+T-1} c(j)P_l(j)$$

$$\begin{aligned} \text{such that} \quad & T(j+1) = \alpha T(j) - \beta R \eta P_l(j) + \beta T_{ex}(j) & \forall j \in [k, k+T-1] \\ & 0 \leq P_l(j) \leq P_l^{nom} & \forall j \in [k, k+T-1] \\ & T(j) \leq T^{sp} + \Delta & \forall j \in [k+1, k+T] \end{aligned}$$

Since we have the forecasts for T_{ex} we set $T_{ex}(j) = \hat{T}_{ex}(j) \forall j \in [k+1: k+T-1]$, whereas for the current time k we use the measured value $T_{ex}(k)$.

Prices $c(j)$ are taken from the ARERA database (file “tariffe-elettricità-non-domestici-2021.xls”)

Model Predictive Control

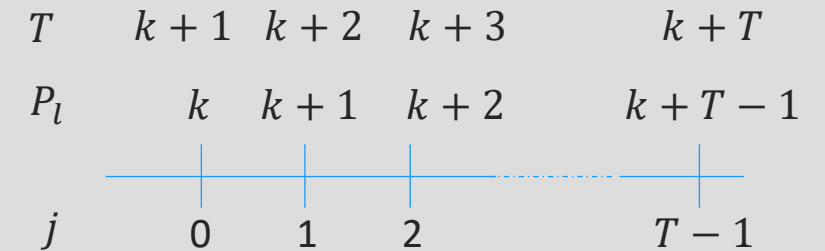
Example 2



The solution is implemented in the MATLAB script “example3.m” and the Python file “example3.py”

FILE example3.py

```
def mpc_step(Tk, Tex_forecast, ck, parms):  
    parms = SimpleNamespace(**parms)  
    model = ConcreteModel() #define the pyomo model  
  
    # Sets  
    model.J = RangeSet(0, parms.T-1) # time index  
  
    # Variables  
    model.Pl = Var(model.J, domain=NonNegativeReals)  
    model.T = Var(model.J, domain=Reals)  
  
    # Define objective function  
    def obj_rule(model):  
        return sum(ck[j]*Pl[j] for j in model.J)  
  
    model.obj = Objective(rule=obj_rule, sense=minimize)  
  
    ...
```



Model Predictive Control

Example 1



The solution is implemented in the MATLAB script “example2.m” and the Python file “example2.py”

FILE example3.py

...

```
# Constraints
def con1(model, j):
    if j==0:
        return model.T[j] == parms.alpha*Tk-parms.beta*parms.R*parms.eta*model.Pl[j]+parms.beta*Tex_forecast[j];
    else:
        return model.T[j] == parms.alpha*model.T[j-1]-parms.beta*parms.R*parms.eta*model.Pl[j]+parms.beta*Tex_forecast[j];
```

```
def con2(model, j):
    return model.Pl[j] <= parms.Pnom
```

```
def con3(model, j):
    return model.T[j] <= parms.Tsp+parms.Delta
```

```
# Add constraints to model
```

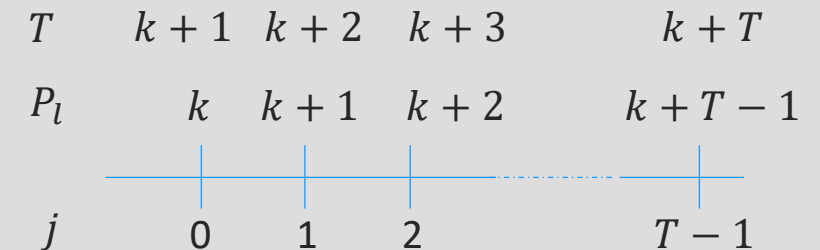
```
model.con_1 = Constraint(model.J, rule=con1)
model.con_2 = Constraint(model.J, rule=con2)
model.con_3 = Constraint(model.J, rule=con3)
```

```
# solve
```

```
results = SolverFactory('gurobi').solve(model)
```

```
out = {"PL":value(model.Pl[0]), #applying receding horizon
       "solver_status": results.solver.status == SolverStatus.ok,
       "opt_condition": results.solver.termination_condition == 'optimal'
}
```

```
return out
```

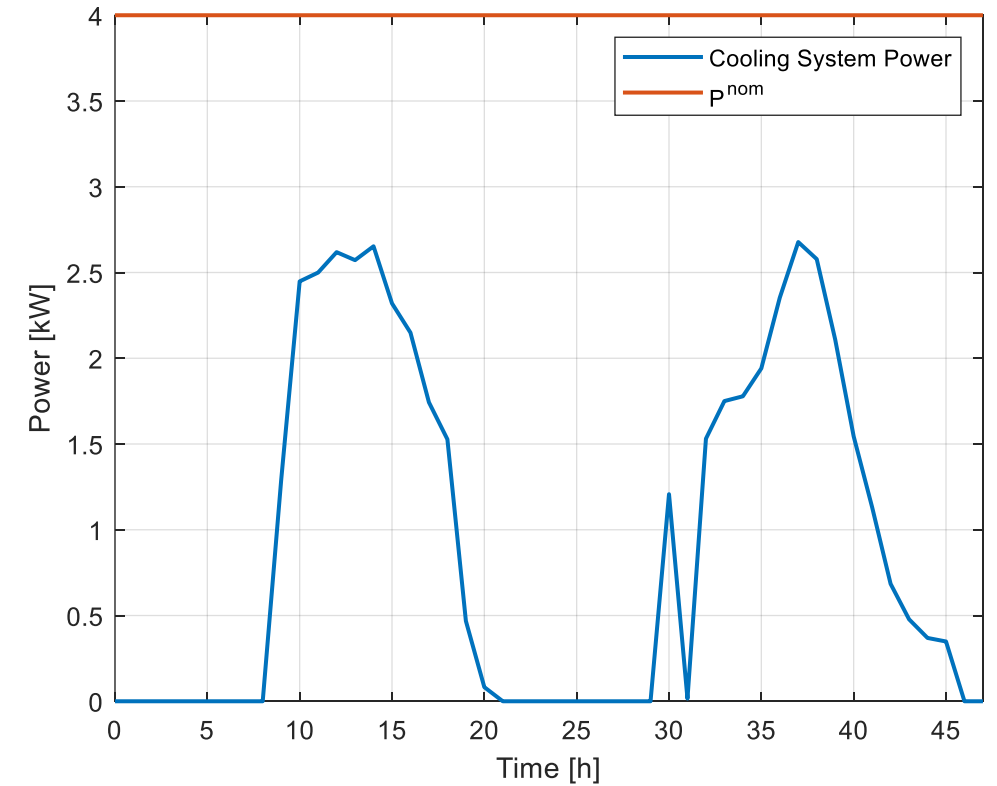
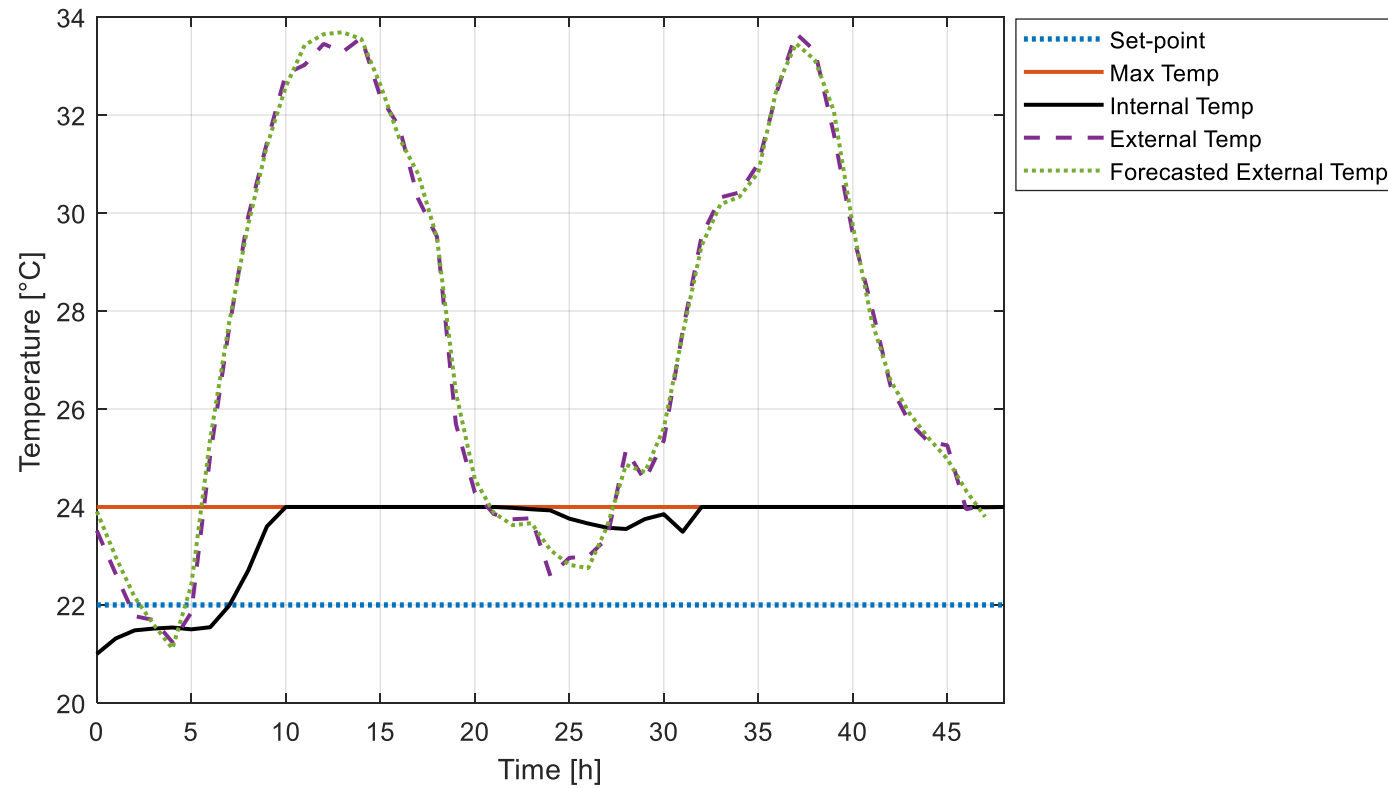


Model Predictive Control

Example 2



Control horizon: $T = 24$

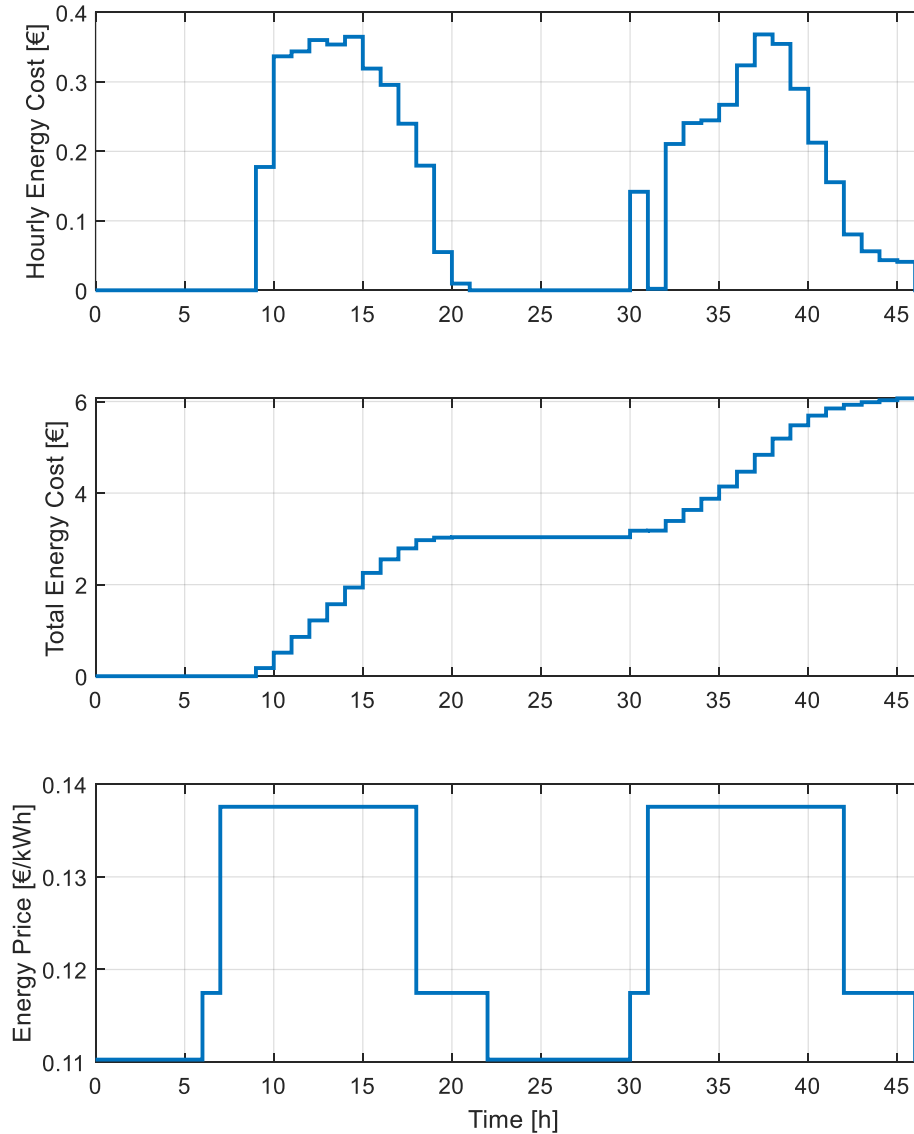


Model Predictive Control

Example 2



Control horizon: $T = 24$



Model Predictive Control

Example 3



Simplified room model for air temperature regulation (heating and cooling)

$$\begin{aligned}T(k+1) &= \alpha T(k) - \beta R(\eta_c P_c(k) - \eta_h P_h(k)) + \beta T_{ex}(k) \\0 &\leq P_c(k) \leq \delta(k) P^{nom} \\0 &\leq P_h(k) \leq (1 - \delta(k)) P^{nom}\end{aligned}$$

Parameters

- $\alpha = \exp\left(-\frac{\Delta t_s}{CR}\right)$ and $\beta = 1 - \alpha$;
- R is the thermal resistance of the walls [$^{\circ}\text{C}/\text{kW}$];
- C is the thermal capacitance of the air [$\text{kWh}/^{\circ}\text{C}$] (energy required to vary the temperature of 1°C);
- $T_{ex}(k)$ external temperature [$^{\circ}\text{C}$] (measured or forecasted);
- η_c cooling COP (>1) [pu];
- η_h heating efficiency (<1) [pu];

Building data: $R = 1.9^{\circ}\text{C}/\text{kW}$, $\eta_c = 1.90$ (COP >1), $\eta_h = 0.85$ (efficiency <1) $C = 3.925$ [$\text{kWh}/^{\circ}\text{C}$]

Control specifications:

Objective: keep the temperature as close as possible to the set-point $T^{sp} = 22^{\circ}\text{C}$ with a tolerance of $\Delta = 2^{\circ}\text{C}$ and minimize the cost of energy consumption.

Assumptions: P_c and P_h can be set with continuity within the interval $[0 \quad P^{nom}]$, with $P^{nom} = 12$ kW;

$\Delta t_s = 1$ h;

$T(k)$ and $T_{ex}(k)$ are measured;

at time k we are provided with a forecast of T_{ex} for the next 24 h;

energy costs at any time $c(k)$ are known.

Model Predictive Control

Example 3



We solve the problem adopting MPC with the following formulation

$$\{P_c^*(j), P_h^*(j)\}_{j=k}^{k+T-1} = \arg \min \sum_{j=k}^{k+T-1} c(j)(P_c(j) + P_h(j))$$

$$\text{such that } T(j+1) = \alpha T(j) - \beta R(\eta_c P_c(j) - \eta_h P_h(j)) + \beta T_{ex}(j) \quad \forall j \in [k, k+T-1]$$

$$0 \leq P_c(j) \leq \delta(j)P^{nom} \quad \forall j \in [k, k+T-1]$$

$$0 \leq P_h(j) \leq (1 - \delta(j))P^{nom} \quad \forall j \in [k, k+T-1]$$

$$T^{sp} - \Delta \leq T(j) \leq T^{sp} + \Delta \quad \forall j \in [k+1, k+T]$$

Since we have the forecasts for T_{ex} we set $T_{ex}(j) = \hat{T}_{ex}(j) \forall j \in [k+1: k+T-1]$, whereas for the current time k we use the measured value $T_{ex}(k)$.

Prices $c(j)$ are taken from the ARERA database (file “tariffe-elettricità-non-domestici-2021.xls”)

Model Predictive Control

Example 3



The solution is implemented in the MATLAB script “example4.m” and the Python file “example4.py”

FILE example4.py

```
def mpc_step(Tk, Tex_forecast, ck, parms):  
    parms = SimpleNamespace(**parms)  
    model = ConcreteModel() #define the pyomo model  
  
    # Sets  
    model.J = RangeSet(0, parms.T-1)    # time index  
  
    # Variables  
    model.Pc = Var(model.J, domain=NonNegativeReals)  
    model.Ph = Var(model.J, domain=NonNegativeReals)  
    model.T = Var(model.J, domain=Reals)  
    model.d = Var(model.J, domain=Binary)  
  
    # Define objective function  
    def obj_rule(model):  
        return sum(ck[j]*(Ph[j]+Pc[j]) for j in model.J)  
  
    model.obj = Objective(rule=obj_rule, sense=minimize)  
  
    ...
```

Model Predictive Control

Example 3



The solution is implemented in the MATLAB script “example4.m” and the Python file “example4.py”

FILE example2.py

...

```
# Constraints
def con1(model, j):
    if j==0:
        return model.T[j] == parms.alpha*Tk-parms.beta*parms.R*(parms.eta_c*model.Pc[j]-parms.eta_h*model.Ph[j])+parms.beta*Tex_forecast[j];
    else:
        return model.T[j] == parms.alpha*model.T[j-1]-parms.beta*parms.R*parms.eta*(parms.eta_c*model.Pc[j]-parms.eta_h*model.Ph[j]) +parms.beta*Tex_forecast[j];

def con2(model, j):
    return model.Pc[j] <= model.d[j]*parms.Pnom

def con3(model, j):
    return model.Ph[j] <= (1-model.d[j])*parms.Pnom

def con4(model, j):
    return model.T[j] <= parms.Tsp+parms.Delta

def con5(model, j):
    return model.T[j] >= parms.Tsp-parms.Delta

# Add constraints to model
model.con_1 = Constraint(model.J, rule=con1)
model.con_2 = Constraint(model.J, rule=con2)
model.con_3 = Constraint(model.J, rule=con3)
model.con_4 = Constraint(model.J, rule=con4)
model.con_5 = Constraint(model.J, rule=con5)

# solve
results = SolverFactory('gurobi').solve(model)

out = {"Pc":value(model.Pc[0]), #applying receding horizon
      "Ph":value(model.Ph[0]), #applying receding horizon
      "solver_status": results.solver.status == SolverStatus.ok,
      "opt_condition": results.solver.termination_condition == 'optimal'
      }

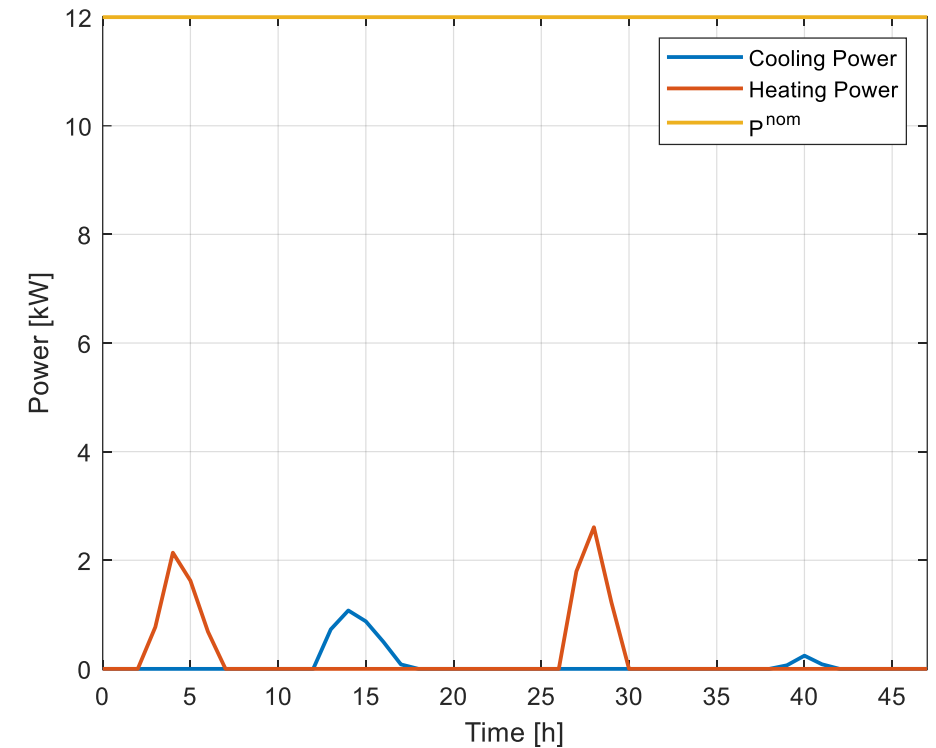
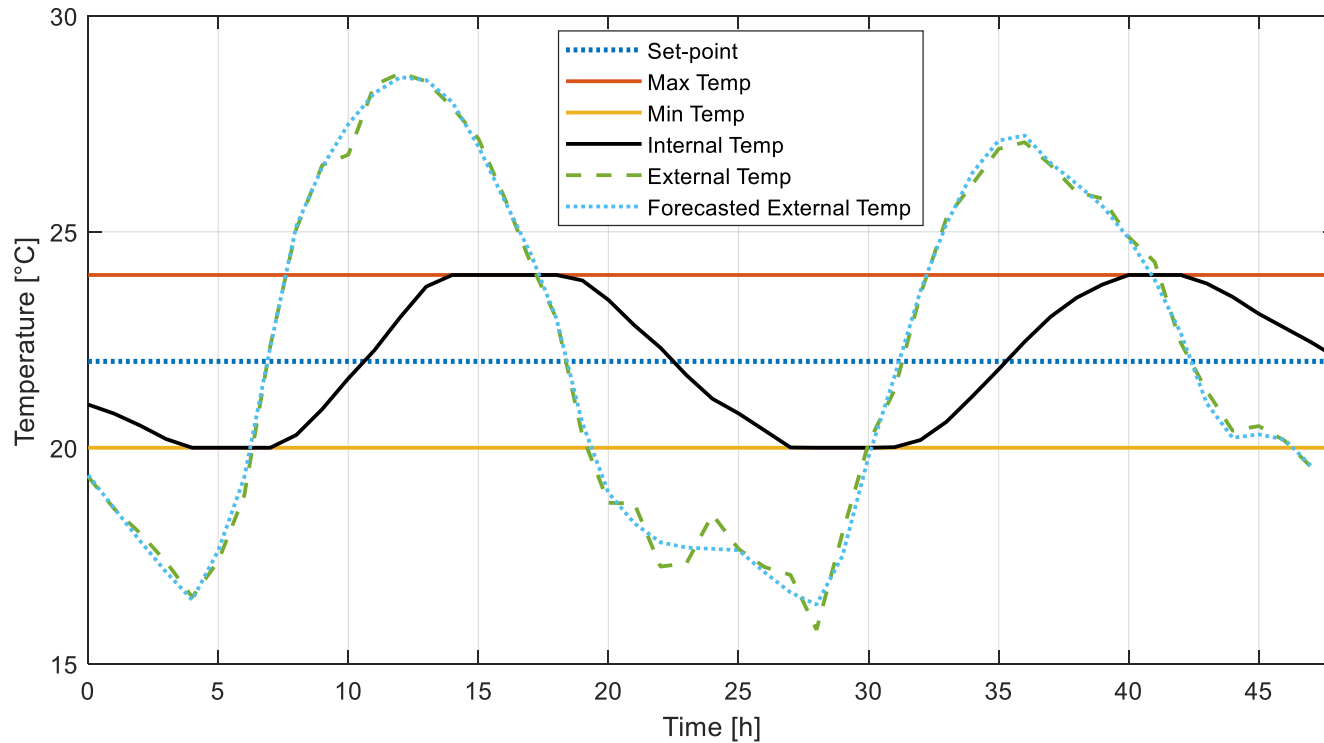
return out
```

Model Predictive Control

Example 3

2021, June 16-17

Control horizon: $T = 24$

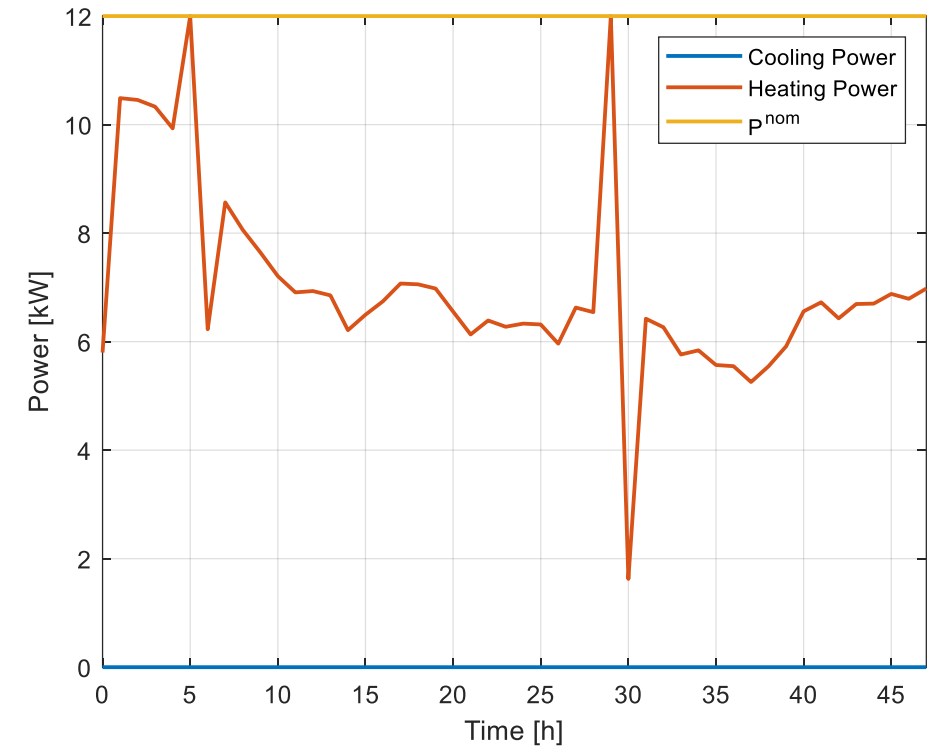
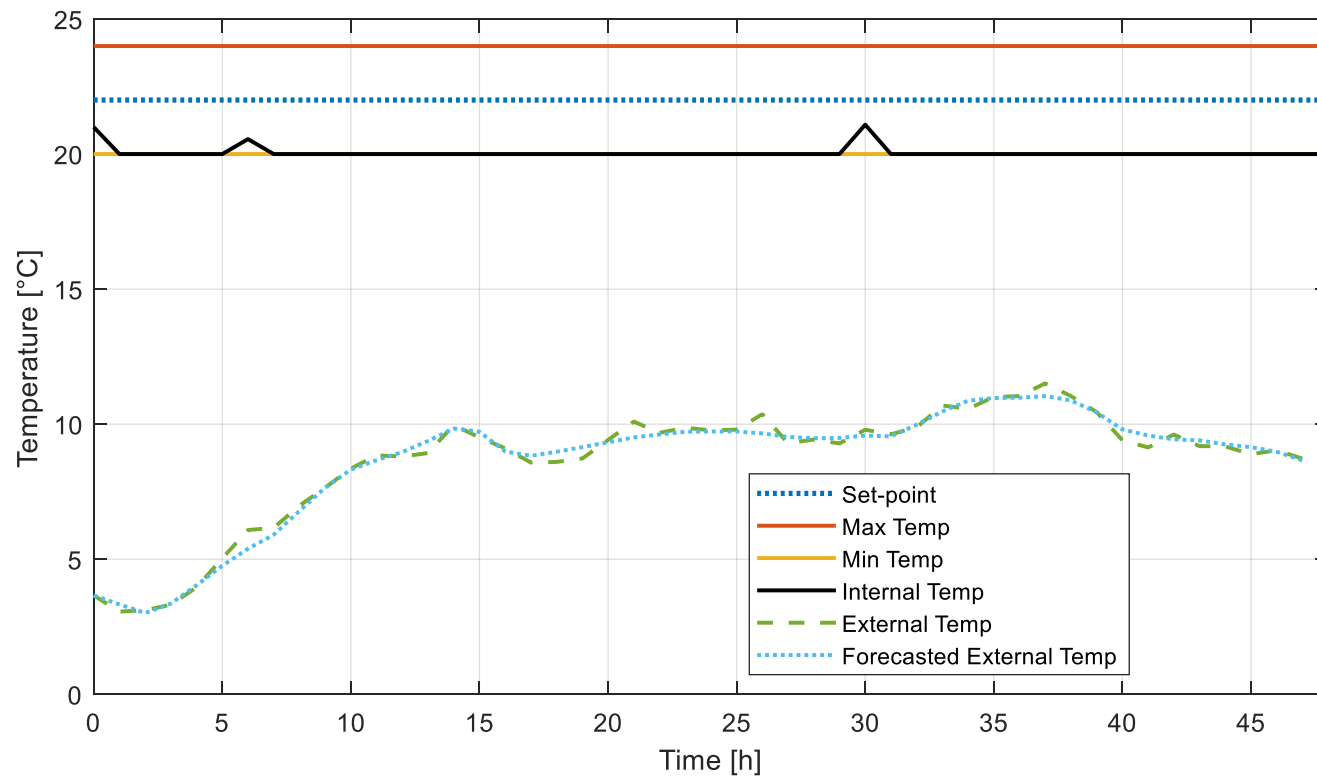


Model Predictive Control

Example 3

2021, January 1-2

Control horizon: $T = 24$

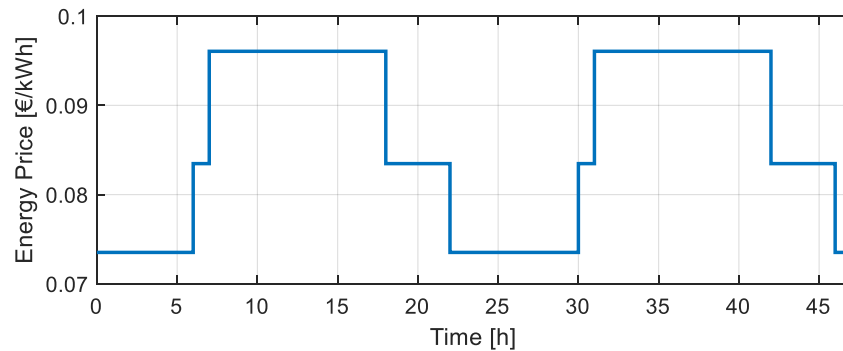
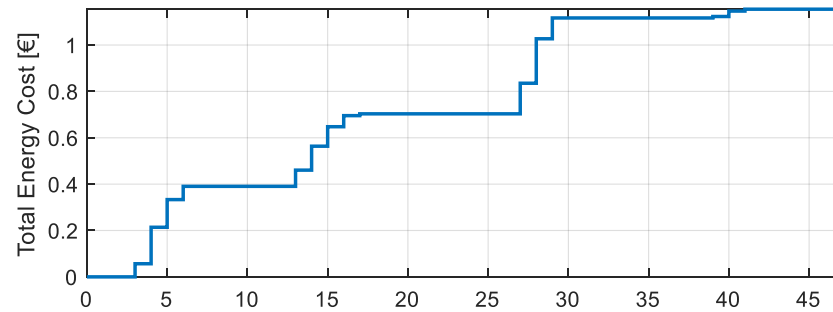
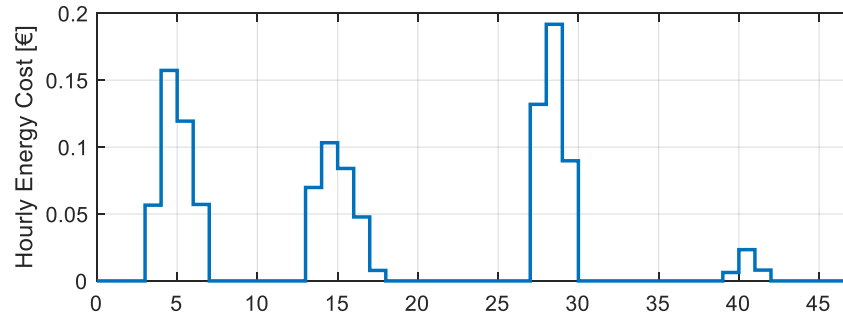


Model Predictive Control

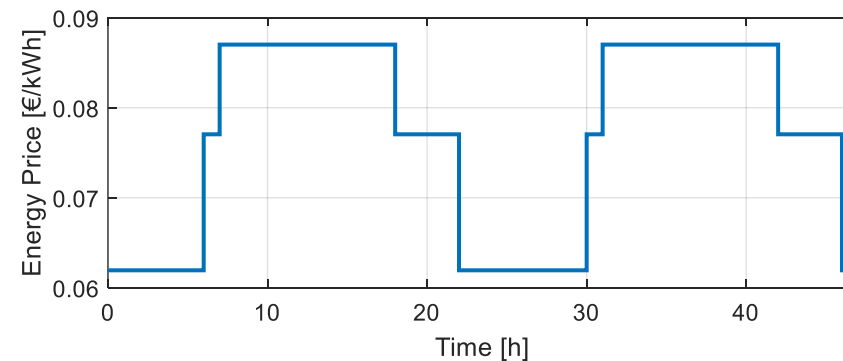
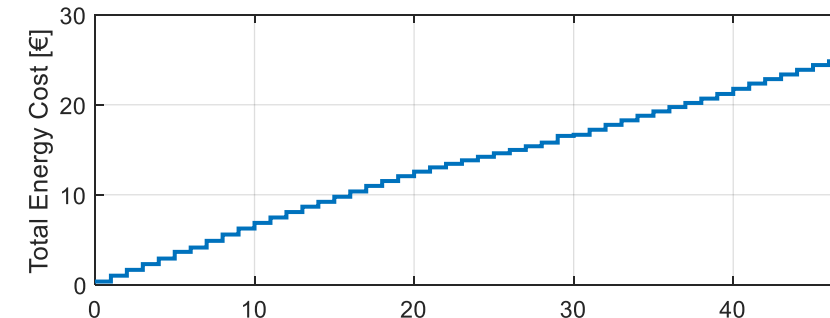
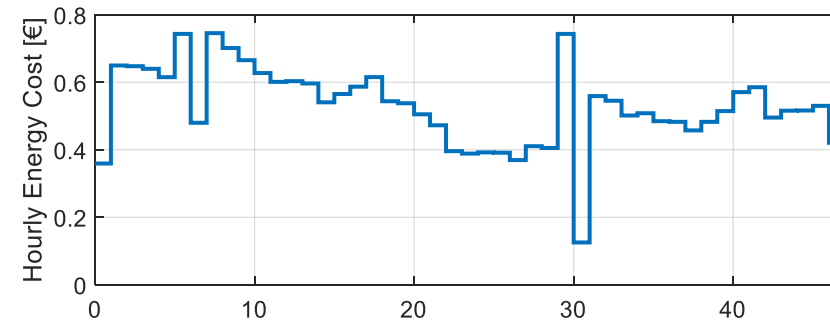
Example 3



2021, June 16-17



2021, January 1-2





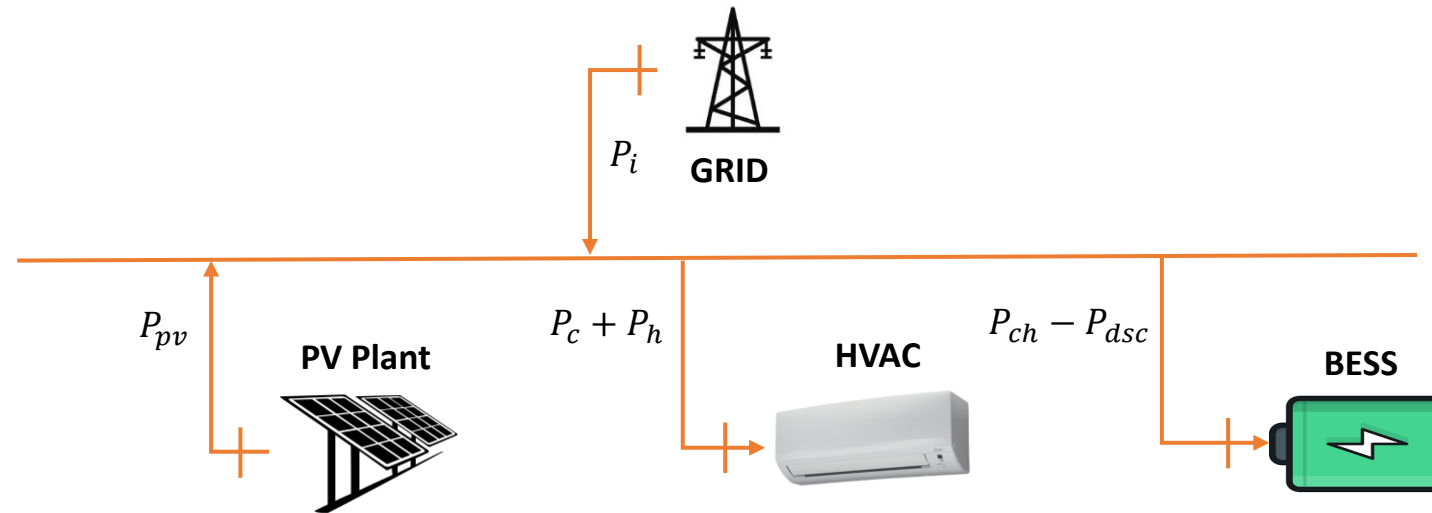
UCBM
UNIVERSITA' CAMPUS BIO-MEDICO DI ROMA



Smart Management Examples

Smart Management Examples

Example 1



35 PV panels
 PV panel area: 1.6275 m²
 Efficiency: 22.2%
 Nominal power: 12.6457 kW

$P_{hvac}^{nom} = 12$ kW
 $R = 1.9$ °C/kW
 $\eta_c = 1.90$ (COP > 1)
 $\eta_h = 0.85$ (efficiency < 1)
 $C = 3.925$ [kWh/°C]

$P_b^{nom} = 8$ kW
 $E_b = 8$ kWh
 $\eta_{ch} = 0.95$
 $\eta_{dsc} = 0.95$

Smart Management Examples

Example 1



Control specifications:

Objective:

- minimize the cost of energy consumption;
- keep the temperature as close as possible to the set-point $T^{sp} = 22^{\circ}\text{C}$ with a tolerance of $\Delta = 2^{\circ}\text{C}$;
- keep the battery State-of-Charge (SoC) within the interval $[0.1 \ 0.9]$.

Assumptions:

- P_c and P_h can be set with continuity within the interval $[0 \ P^{nom}]$, with $P^{nom} = 12 \text{ kW}$;
- $\Delta t_s = 1 \text{ h}$;
- $T(k)$, $T_{ex}(k)$, $P_{pv}(k)$, $SoC(k)$ are measured at time k ;
- at time k we are provided with a forecast of the external temperature T_{ex} for the next 24 h;
- at time k we are provided with a forecast of the PV generation P_{pv} for the next 24 h;
- energy costs at any time $c(k)$ are known (ARERA working day prices are assumed - file “tariffe-elettricità-non-domestici-2021.xls”);
- PV generation self-consumption is paid as a tariff discount c_{td}^{scpv} (set to 8.22 €/MWh).

Smart Management Examples

Example 1



We solve the problem adopting MPC with the following formulation

$$\{P_c^*(j), P_h^*(j), P_{ch}^*(j), P_{dsc}^*(j), P_{curt}^*(j)\}_{j=k}^{k+T-1} = \arg \min \sum_{j=k}^{k+T-1} (c(j)P_i(j) + c_{td}^{scpv} P_{curt}(j))$$

HVAC system model	such that	$T(j+1) = \alpha T(j) - \beta R(\eta_c P_c(j) - \eta_h P_h(j)) + \beta T_{ex}(j)$	$\forall j \in [k, k+T-1]$
		$0 \leq P_c(j) \leq \delta_{hvac}(j) P_{hvac}^{nom}$	$\forall j \in [k, k+T-1]$
		$0 \leq P_h(j) \leq (1 - \delta_{hvac}(j)) P_{hvac}^{nom}$	$\forall j \in [k, k+T-1]$
		$T^{sp} - \Delta \leq T(j) \leq T^{sp} + \Delta$	$\forall j \in [k+1, k+T]$
Battery model		$SoC(j+1) = SoC(j) + \frac{\Delta t_s}{E_b} \left(\eta_{ch} P_{ch}(j) - \frac{1}{\eta_{dsc}} P_{dsc}(j) \right)$	$\forall j \in [k, k+T-1]$
		$0 \leq P_{ch}(j) \leq \delta_b(j) P_b^{nom}$	$\forall j \in [k, k+T-1]$
		$0 \leq P_{dsc}(j) \leq (1 - \delta_b(j)) P_b^{nom}$	$\forall j \in [k, k+T-1]$
		$SoC^{min} \leq SoC(j) \leq SoC^{max}$	$\forall j \in [k+1, k+T]$
System constraints		$P_i(j) = P_c(j) + P_h(j) + P_{ch}(j) - P_{dsc}(j) - (P_{pv}(j) - P_{curt}(j))$	$\forall j \in [k, k+T-1]$
		$P_i(j) \geq 0$	$\forall j \in [k, k+T-1]$
		$0 \leq P_{curt}(j) \leq P_{pv}(j)$	$\forall j \in [k, k+T-1]$

- $T_{ex}(j) = \hat{T}_{ex}(j) \forall j \in [k+1: k+T-1]$, whereas for the current time k we use the measured value $T_{ex}(k)$
- $P_{pv}(j) = \hat{P}_{pv}(j) \forall j \in [k: k+T-1]$
- $SoC(k), T(k)$ measured

Smart Management Examples

Example 1

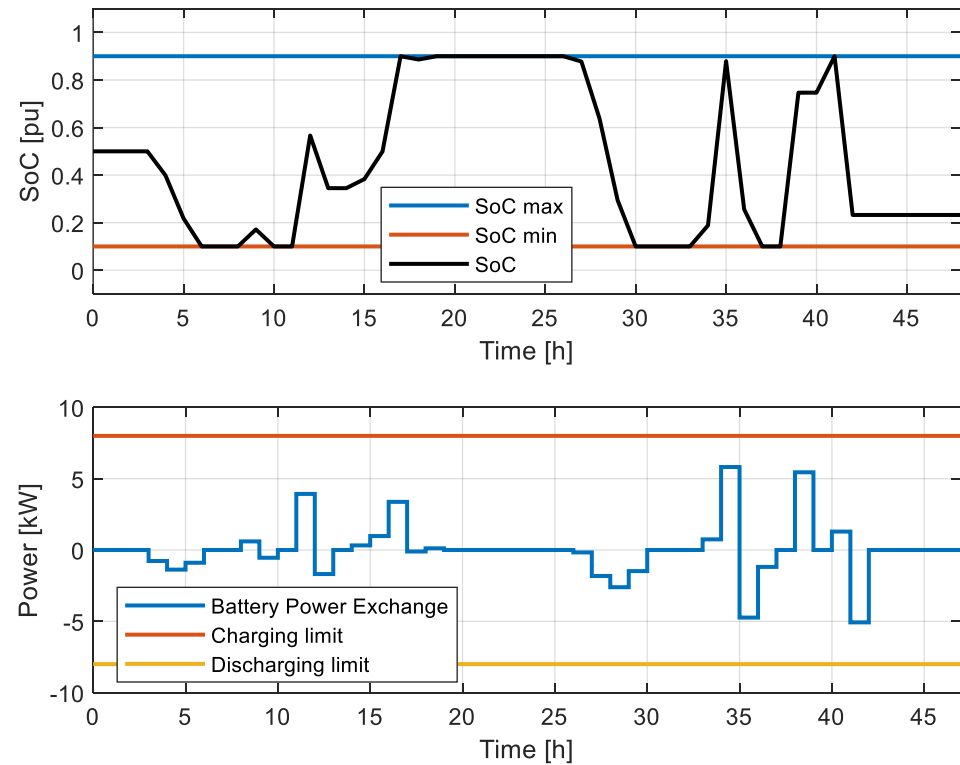
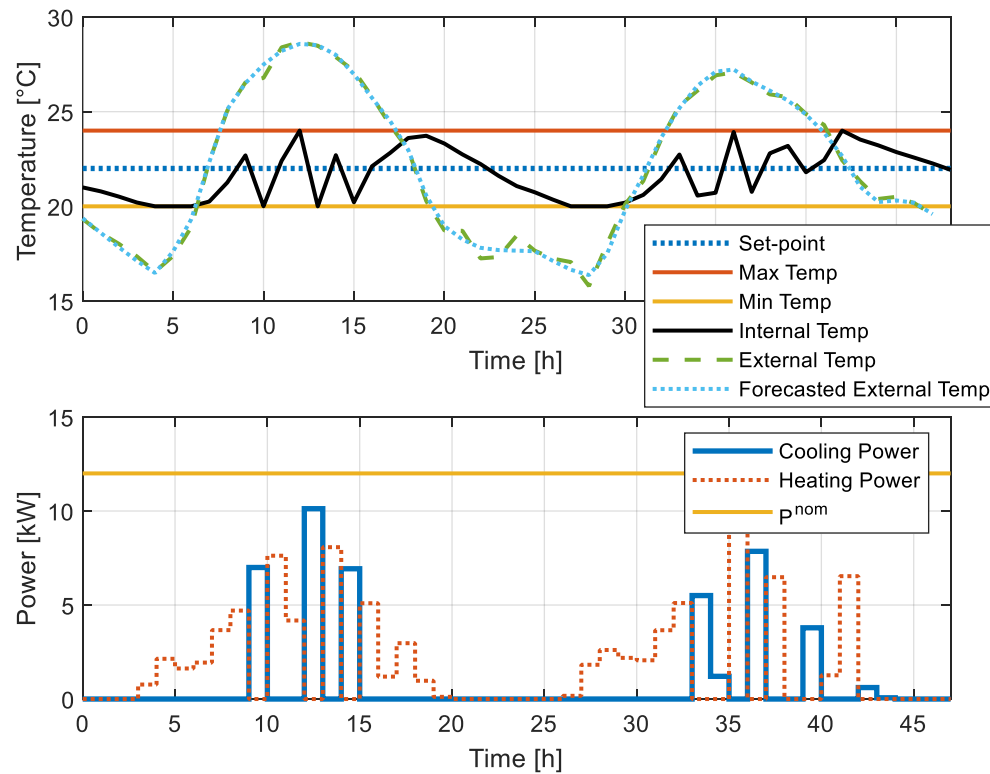


The solution is implemented in the MATLAB script “example5.m” and the Python model file “example5.py”

Smart Management Examples

Example 1

Control horizon: $T = 24$
 2021 June 15-16

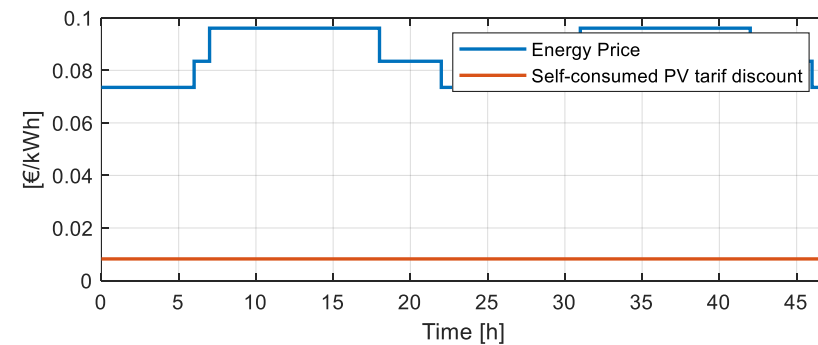
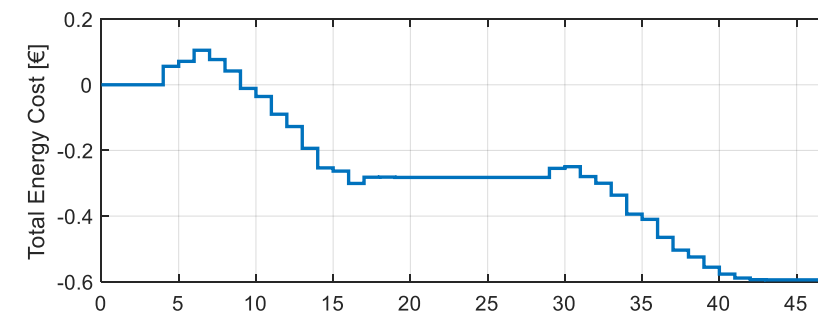
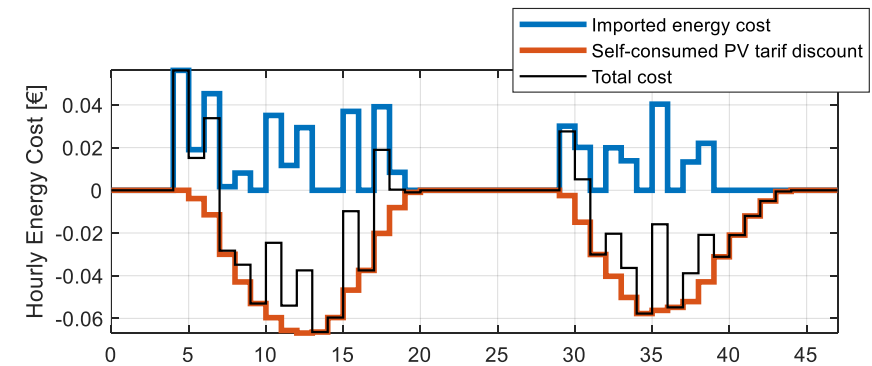
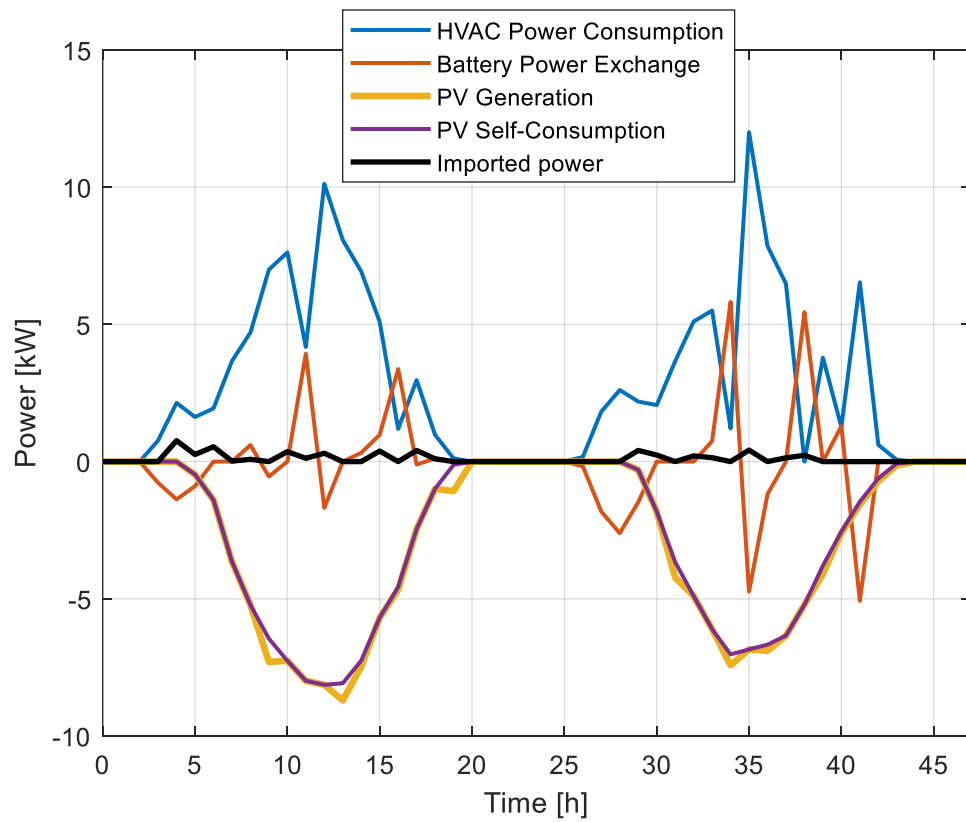


Smart Management Examples

Example 1

Control horizon: $T = 24$

2021 June 15-16

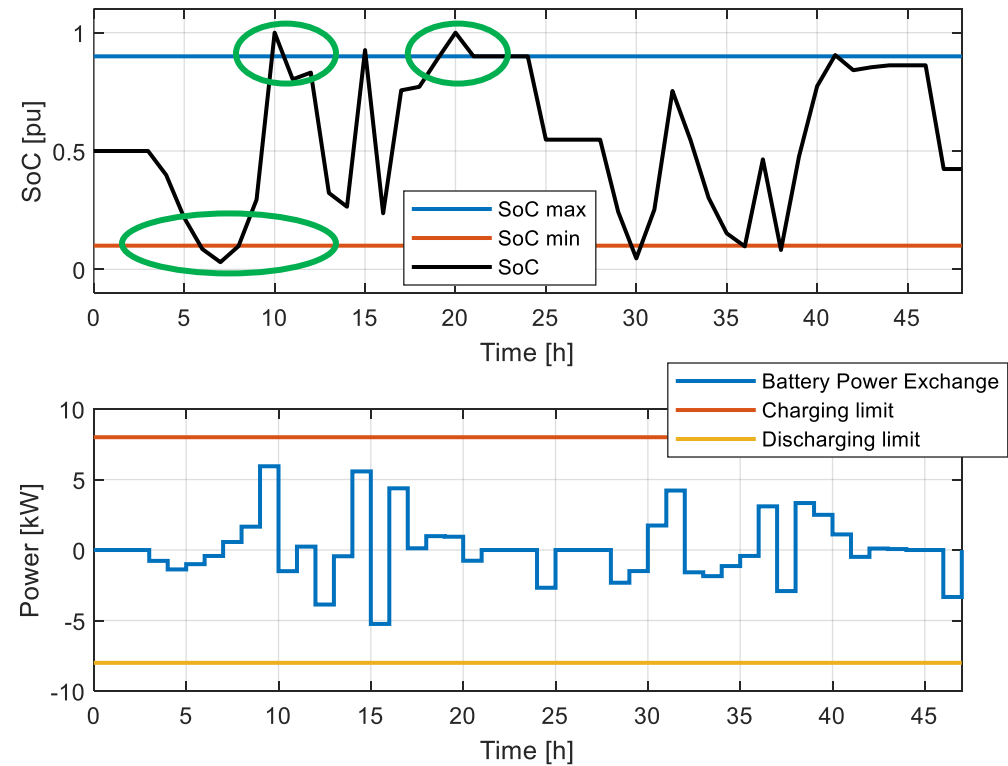
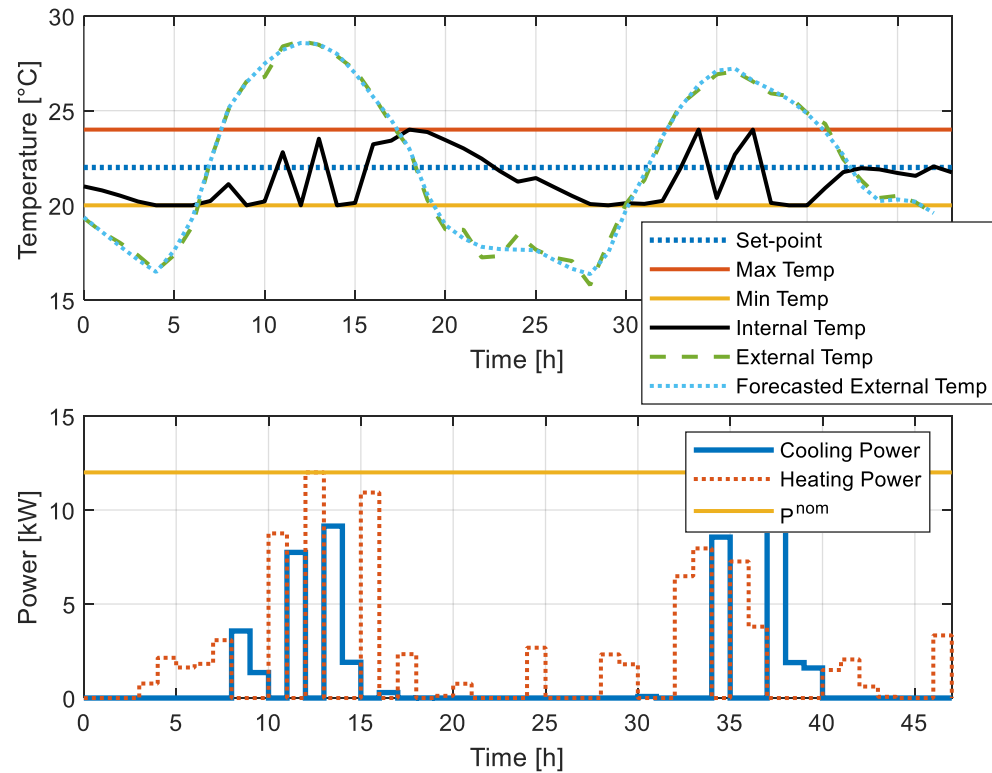


Smart Management Examples

Example 1



Control horizon: $T = 24$ **WITH ERROR COMPENSATION BY BATTERY**
2021 June 15-16

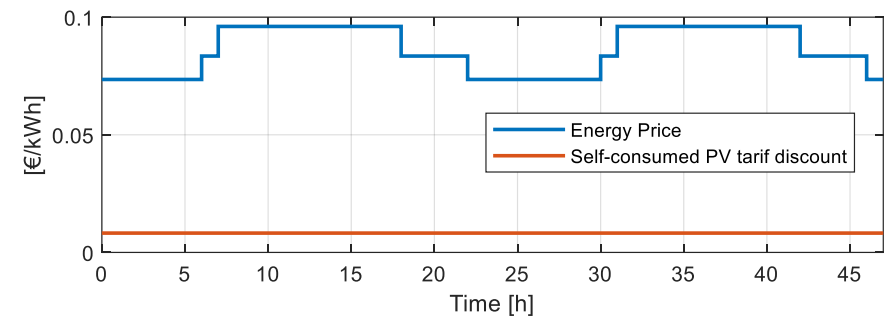
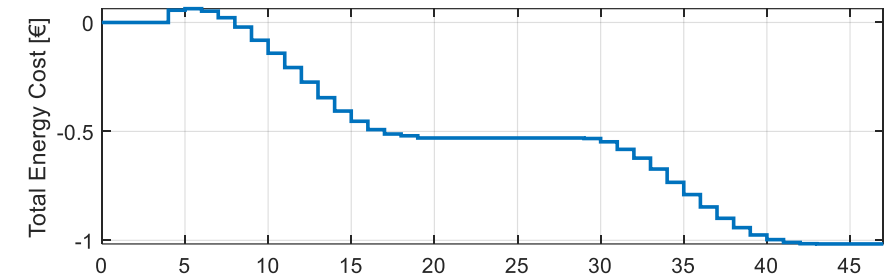
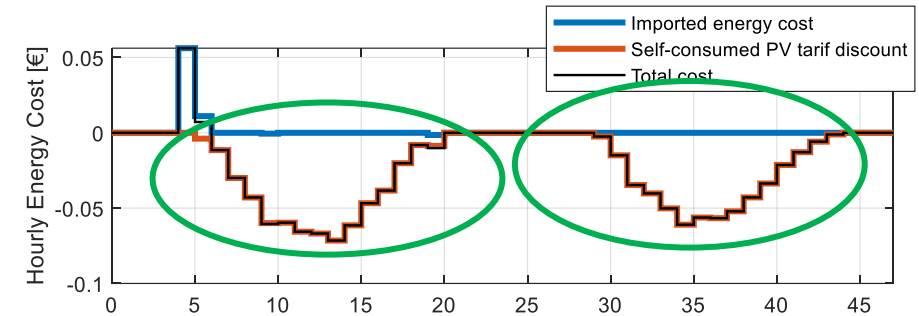
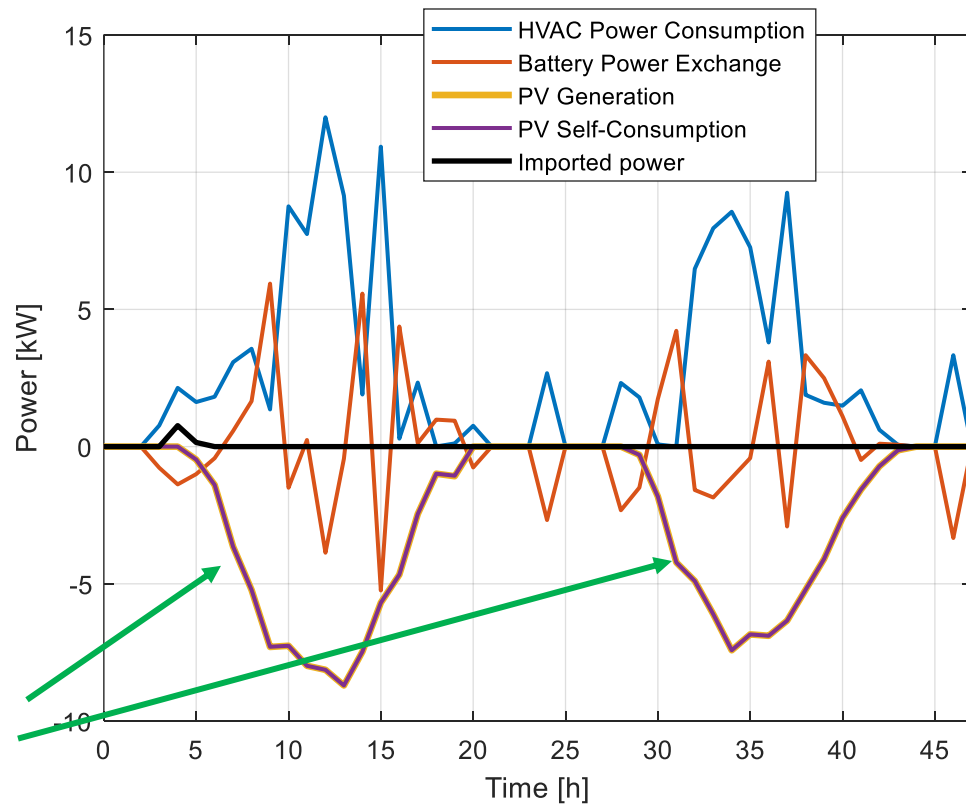


Smart Management Examples

Example 1

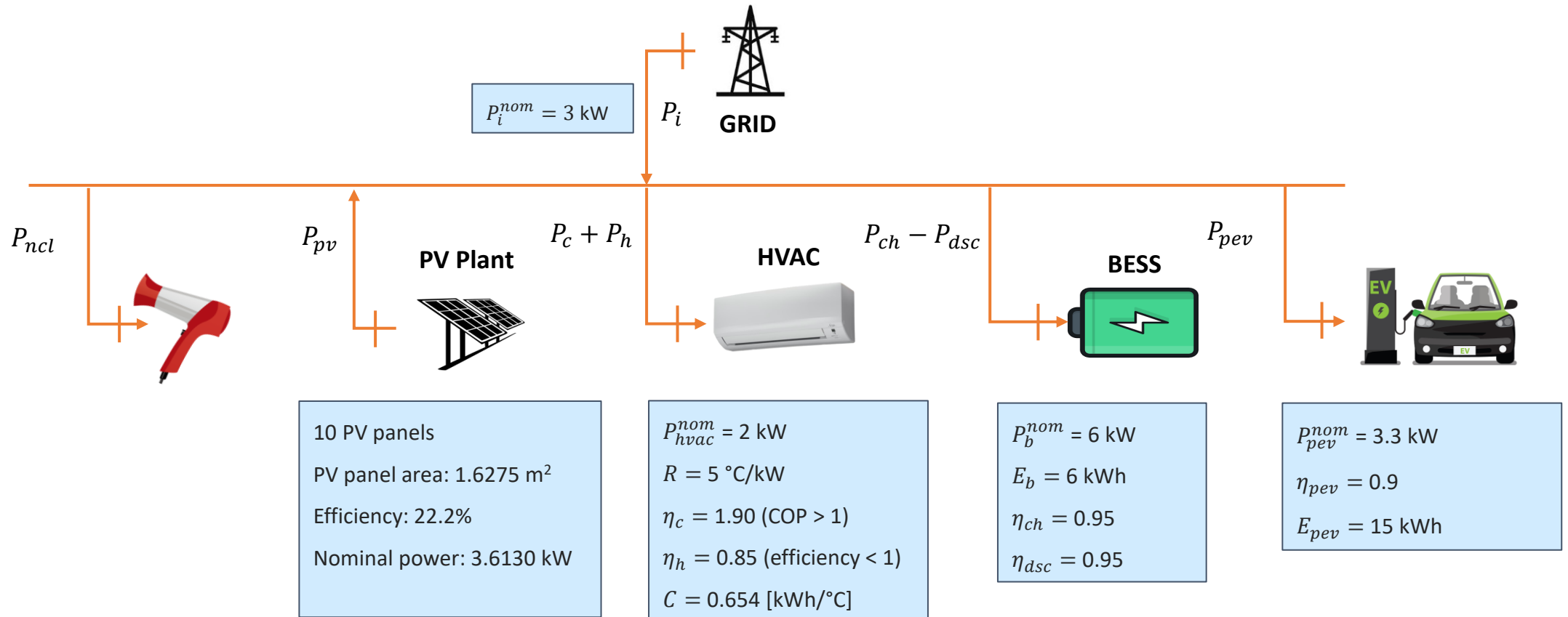


Control horizon: $T = 24$ **WITH ERROR COMPENSATION BY BATTERY**
2021 June 15-16



Smart Management Examples

Example 2 (Smart Home)



Smart Management Examples

Example 2 (Smart Home)



Control specifications:

Objective:

- satisfy the user requirements;
- minimize the cost of energy consumption;
- keep the temperature as close as possible to the set-point T^{sp} indicated by the user with a tolerance of $\Delta = 2^\circ\text{C}$;
- keep the battery State-of-Charge (SoC) within the interval $[0.1 \ 0.9]$.

Assumptions:

- P_c and P_h can be set with continuity within the interval $[0 \ P_{hvac}^{nom}]$;
- $\Delta t_s = 1 \text{ h}$;
- $T(k), T_{ex}(k), P_{pv}(k), SoC(k), SoC_{pev}(k)$ are measured at time k ;
- at time k we are provided with a forecast of the external temperature T_{ex} for the next 24 h;
- at time k we are provided with a forecast of the PV generation P_{pv} for the next 24 h;
- at time k we are provided with a forecast of the UCL P_{ncl} for the next 24 h;
- energy costs at any time $c(k)$ are known (ARERA working day prices are assumed - file “tariffe-elettricit -non-domestici-2021.xls”);
- PV generation self-consumption is paid as a tariff discount c_{td}^{scpv} (set to 8.22 €/MWh);
- user time requirements are declared (see next slide)

Smart Management Examples

Example 2 (Smart Home)



User time requirements are declared:

- for controllable load (PEV, HVAC), the user indicates in advance, the time interval within which the load can be switched on by setting to 1 $UR_{pev}(j)$, $UR_{hvac}(j)$, if the device can be switched on at the hour j ;

example: we are at time k and we want that the PEV is recharged in 6 hours:

$$UR_{pev}(j) = 1 \quad \forall j = k, k + 1, k + 6 - 1$$

$$UR_{pev}(j) = 0 \quad \forall j = k + 6, k + 7, \dots, k + 24 - 1$$

which means:

$$UR_{pev} = [1 \quad 1 \quad 1 \quad 1 \quad 1 \quad 1 \quad 1 \quad 0 \quad \dots \quad 0]$$

- for PEV the user declares the SoC to be reached: SoC_{pev}^* ;
- for HVAC, the user declares the temperature set point $T^{sp}(j)$ for each time interval in the next 24 hours;

Smart Management Examples

Example 2 (Smart Home)



We solve the problem adopting MPC with the following formulation

$$\{P_c^*(j), P_h^*(j), P_{ch}^*(j), P_{dsc}^*(j), P_{curt}^*(j), P_{pev}^*(j)\}_{j=k}^{k+T-1} = \arg \min \sum_{j=k}^{k+T-1} (c(j)P_i(j) + c_{td}^{scpv}P_{curt}(j) + \gamma\epsilon(j))$$

Relaxing
variables

HVAC system model	{	$T(j+1) = \alpha T(j) - \beta R(\eta_c P_c(j) - \eta_h P_h(j)) + \beta T_{ex}(j)$	$\forall j \in [k, k+T-1]$
		$0 \leq P_c(j) \leq \delta_c(j)P_{hvac}^{nom}$	$\forall j \in [k, k+T-1]$
		$0 \leq P_h(j) \leq \delta_h(j)P_{hvac}^{nom}$	$\forall j \in [k, k+T-1]$
		$(T^{sp}(j) - \Delta - \epsilon(j)) \cdot UR_{hvac}(j) \leq T(j) \cdot UR_{hvac}(j) \leq (T^{sp}(j) + \Delta + \epsilon(j)) \cdot UR_{hvac}(j)$	$\forall j \in [k+1, k+T]$
		$\epsilon(j) \geq 0$	$\forall j \in [k+1, k+T]$
		$\delta_c(k) + \delta_h(j) \leq UR_{hvac}(j)$	$\forall j \in [k+1, k+T]$
Battery model	{	$SoC(j+1) = SoC(j) + \frac{\Delta t_s}{E_b} \left(\eta_{ch} P_{ch}(j) - \frac{1}{\eta_{dsc}} P_{dsc}(j) \right)$	$\forall j \in [k, k+T-1]$
		$0 \leq P_{ch}(j) \leq \delta_b(j)P_b^{nom}$	$\forall j \in [k, k+T-1]$
		$0 \leq P_{dsc}(j) \leq (1 - \delta_b(j))P_b^{nom}$	$\forall j \in [k, k+T-1]$
		$SoC^{min} \leq SoC(j) \leq SoC^{max}$	$\forall j \in [k+1, k+T]$
PEV model	{	$\Delta t_s \sum_{j=k}^{k+T-1} \eta_{pev} P_{pev}(j) = E_{pev} \cdot (SoC_{pev}^* - SoC_{pev}(k))$	
		$0 \leq P_{pev}(j) \leq UR_{pev}(j)P_{pev}^{nom}$	$\forall j \in [k, k+T-1]$

Smart Management Examples

Example 2 (Smart Home)



We solve the problem adopting MPC with the following formulation

$$\{P_c^*(j), P_h^*(j), P_{ch}^*(j), P_{dsc}^*(j), P_{curt}^*(j), P_{pev}^*(j)\}_{j=k}^{k+T-1} = \arg \min \sum_{j=k}^{k+T-1} (c(j)P_i(j) + c_{td}^{scpv}P_{curt}(j) + \gamma\epsilon(j))$$

$$\text{System constraints} \quad \dots \quad \left\{ \begin{array}{l} P_i(j) = P_c(j) + P_h(j) + P_{ch}(j) - P_{dsc}(j) - (P_{pv}(j) - P_{curt}(j)) + P_{ncl}(j) + P_{pev}(j) \\ 0 \leq P_i(j) \leq P_i^{nom} \\ P_{curt}(j) \leq P_{pv}(j) \end{array} \right. \quad \begin{array}{l} \forall j \in [k, k+T-1] \\ \forall j \in [k, k+T-1] \\ \forall j \in [k, k+T-1] \end{array}$$

- $T_{ex}(j) = \hat{T}_{ex}(j) \forall j \in [k+1: k+T-1]$, whereas for the current time k we use the measured value $T_{ex}(k)$
- $P_{pv}(j) = \hat{P}(j) \forall j \in [k: k+T-1]$
- $SoC(k), T(k), SoC_{pev}(k)$ measured
- $P_{ncl}(j) = \hat{P}_{ncl}(j) \forall j \in [k: k+T-1]$

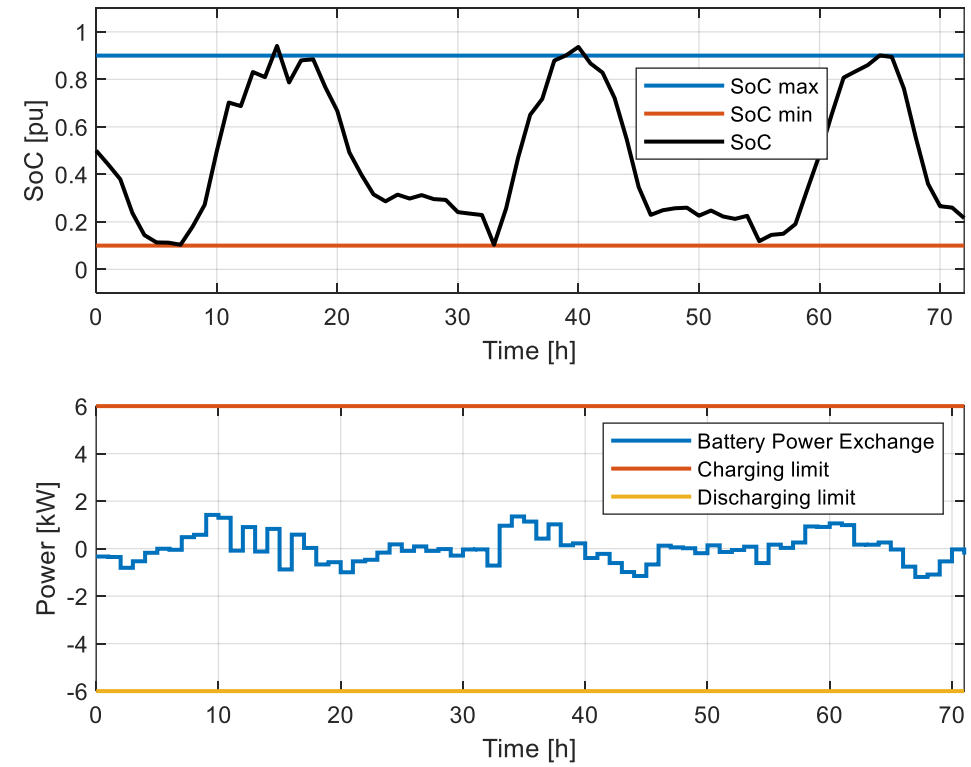
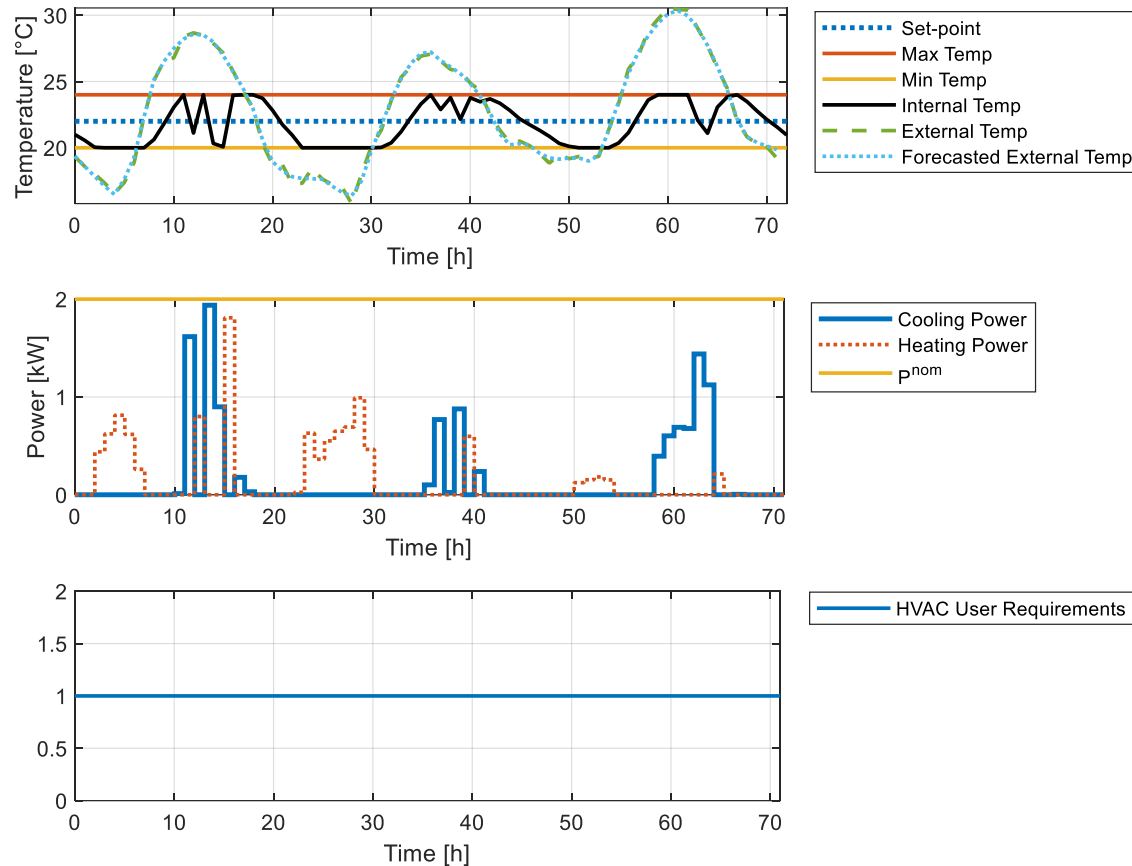
The solution is implemented in the MATLAB script “example6.m” and the Python file “example6.py”

Smart Management Examples

Example 2 (Smart Home)



Control horizon: $T = 24$ **WITH ERROR COMPENSATION BY BATTERY**
2021 June 15-17

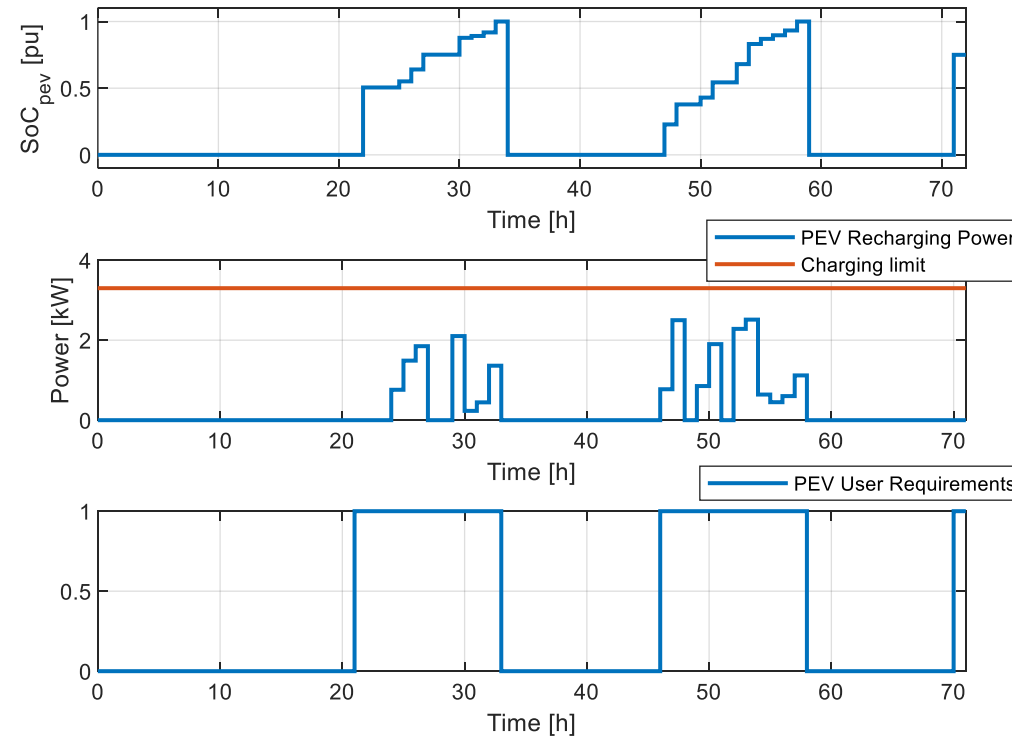


Smart Management Examples

Example 2 (Smart Home)



Control horizon: $T = 24$ **WITH ERROR COMPENSATION BY BATTERY**
2021 June 15-17

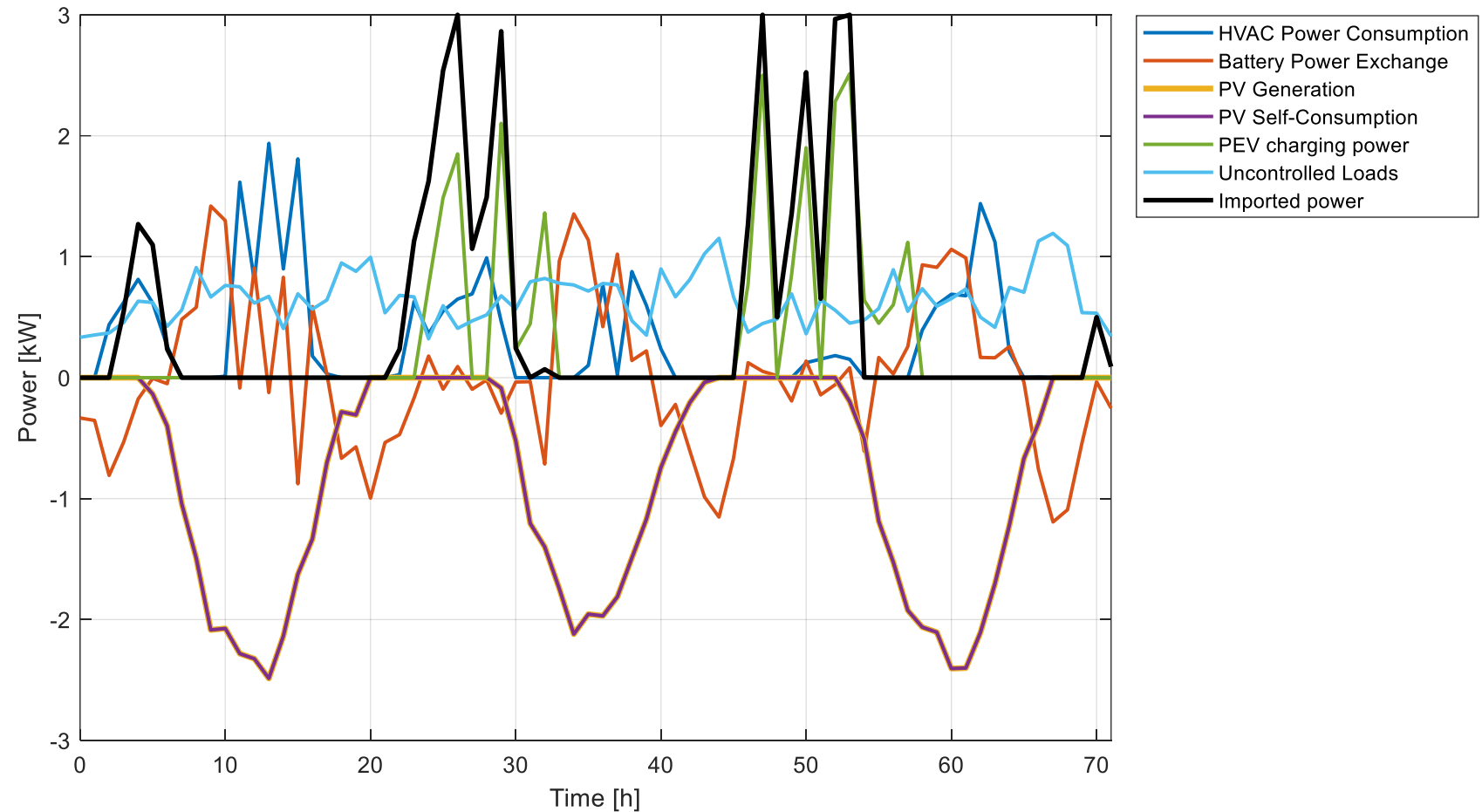


Smart Management Examples

Example 2 (Smart Home)



Control horizon: $T = 24$ **WITH ERROR COMPENSATION BY BATTERY**
2021 June 15-17



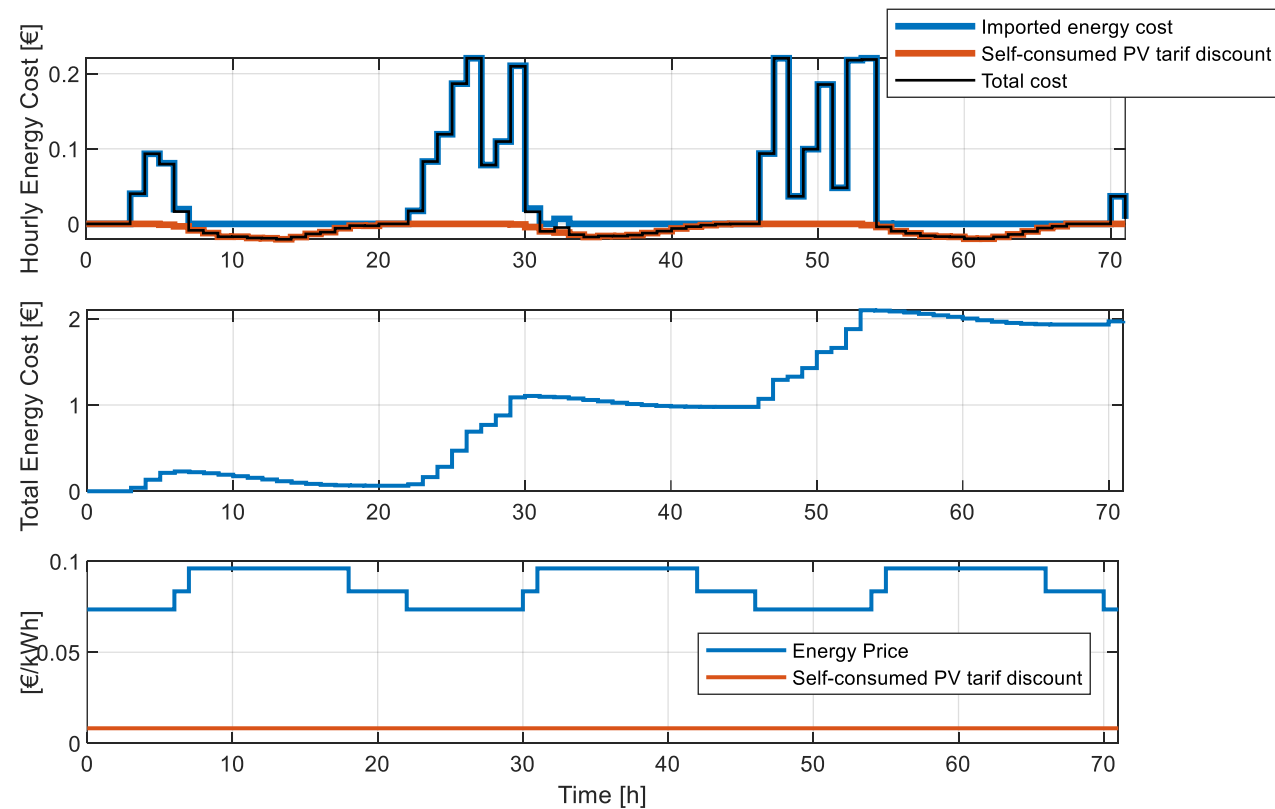
Smart Management Examples

Example 2 (Smart Home)



Control horizon: $T = 24$ **WITH ERROR COMPENSATION BY BATTERY**

2021 June 15-17





Innovazione e trasformazione digitale

f.conte@unicampus.it